PUBLIC

Code Assessment

of the sPOL Smart Contracts

February 24, 2026

Produced for

by

Contents

1 Executive Summary

Dear Polygon Team,

Thank you for trusting us to help Polygon with this security audit. Our executive summary provides an overview of subjects covered in our audit of the latest reviewed contracts of sPOL according to Scope to support you in forming an opinion on their security risks.

Polygon implements sPOL, a liquid staking wrapper for POL that operates across Ethereum and Polygon PoS. Users can mint and burn sPOL on L1 against delegated POL, or buy and sell on L2 with regular reconciliation via cross-chain migration and backfill flows.

The most critical subjects covered in our audit are asset solvency, functional correctness, and cross-chain accounting. Security regarding all the aforementioned subjects is high, since all related high-severity findings were fixed, including sPOLChild Missing fallback to receive POL, Missing polBalance decrease during migration, Fund lockup due to inconsistent validation in sellSPOL and withdrawPOL, and L1 Exchange-Rate Appreciation Can Brick L2 Migrations or Enable Arbitrage.

Residual risk remains primarily in liveness edge cases, where Locked or disabled validators break sPOL flows is only partially corrected. First Depositor Inflation Attack was addressed at the specification level via explicit trust-model assumptions on initial supply seeding and locking.

The general subjects covered include integration with Polygon staking and the bridge stack, as well as trust assumptions around admin actions. While security regarding integrations is high, admin-accessible functions still require careful sequencing and operational safeguards to avoid adverse outcomes.

Given the identified integration issues in the bridge stack, the testing framework would benefit from incorporating real bridge operations alongside mocks to improve coverage of cross-chain flows and edge cases.

In summary, we find that the codebase provides a good level of security.

It is important to note that security audits are time-boxed and cannot uncover all vulnerabilities. They complement but don't replace other vital measures to secure a project.

The following sections will give an overview of the system, our methodology, the issues uncovered, and how they have been addressed. We are happy to receive questions and feedback to improve our service.

Sincerely yours,

ChainSecurity

1.1 Overview of the Findings

Below we provide a brief numerical overview of the findings and how they have been addressed.

Critical-Severity Findings	0
High-Severity Findings	4
• Code Corrected	4
Medium-Severity Findings	1
• Code Partially Corrected	1
Low-Severity Findings	12
• Code Corrected	6
• Specification Changed	2
• Risk Accepted	4

2 Assessment Overview

In this section, we briefly describe the overall structure and scope of the engagement, including the code commit which is referenced throughout this report.

2.1 Scope

The assessment was performed on the source code files inside the sPOL repository based on the documentation files. The table below indicates the code versions relevant to this report and when they were received.

V	Date	Commit Hash	Note
1	12 January 2026	1626485eee367c7ebae5482ecfb4a2b499390d09	Initial Version
2	11 February 2026	eef7fc164a77d7d79e8b3fb939e30cfdbe7ad902	Fixes
3	13 February 2026	8add3b4e59d1acff585453369743ec717c4671af	Updated Fixes
4	23 February 2026	8d15bf5316f2b8025268294bc17b47308dd31ec5	Fixes

For the Solidity smart contracts, the compiler version `0.8.30` was chosen, targeting EVM hard fork `Prague`.

The following components were included in the scope of this review:

```
src/MsgCoder.sol
src/polBridger.sol
src/sPOL.sol
src/sPOLChild.sol
src/sPOLController.sol
src/sPOLMessenger.sol
```

2.1.1 Excluded from scope

Anything not explicitly listed in the scope section above is considered out of scope for this assessment. This includes, but is not limited to the `src/msg` directory, systems integrated with, tests, and deployment scripts. In the context of this assessment, the message passing system is considered out of scope.

2.2 System Overview

This system overview describes the initially received version (Version 1) of the contracts as defined in the Assessment Overview.

At the end of this report section, we have added subsections for each of the changes according to the versions.

Furthermore, in the findings section, we have added a version icon to each of the findings to increase the readability of the report.

Polygon implements sPOL, a liquid staking token wrapper around POL. The system operates across Ethereum and the Polygon PoS chain. When users deposit POL, they receive sPOL representing their

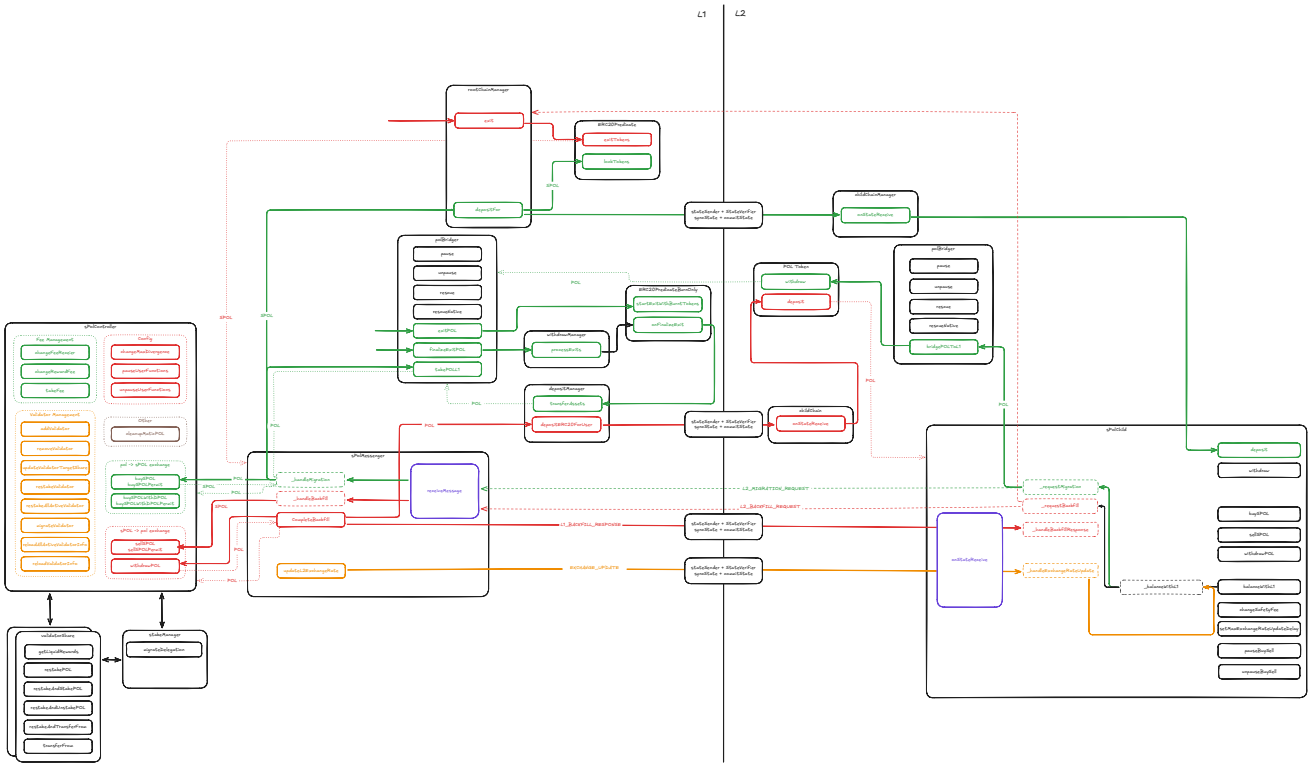
share of the total staked position. The underlying POL is delegated to Polygon PoS validators, earning staking rewards.

The core staking logic resides on Ethereum. The `sPOLController` manages validator delegation and reward restaking, and controls sPOL minting and burning. On Polygon PoS, the `sPOLChild` contract allows users to buy and sell sPOL using native POL, with local accounting that is regularly reconciled with Ethereum. The `sPOLMessenger` on Ethereum coordinates cross-chain communication, relaying exchange rate updates to L2 and processing migration and backfill requests.

Users can interact with the system on either chain. L1 operations interact directly with validators, while L2 operations are batched and settled with L1 through migration (staking surplus POL) or backfill (unstaking POL to cover withdrawals).

In the following, we refer to Ethereum Mainnet as L1 (or the *Root* chain) and Polygon PoS as L2 (or the *Child* chain). Note that Polygon PoS is a sidechain with its own consensus; "L2" is used for convenience. We may write "Polygon" for "Polygon PoS". The native token on L2 is POL.

The following diagram illustrates the high-level architecture and interactions between the main contracts across both chains according to the latest version of the system review:



The following sections provide background on cross-chain state propagation between Ethereum and Polygon PoS, followed by detailed descriptions of the `sPOLController`, `sPOLChild`, and `sPOLMessenger` contracts, and finally the token bridging mechanisms that underpin the system.

2.2.1 Cross-Chain State Propagation: Ethereum ↔ Polygon PoS

State propagation between Ethereum and Polygon PoS is asymmetric and relies on two distinct primitives:

- **Ethereum → Polygon (state sync)** Messages originating on Ethereum are delivered via Polygon's native state sync. On Ethereum, the `StateSender` contract emits a `StateSynced` event for a specified `receiver`. Polygon validators relay that event into the PoS chain, where the `StateReceiver` (StateSyncer) executes it by calling `onStateReceive()` on the target contract as a system call. If execution fails, the message is retained in `StateReceiver` and can be retried

by anyone via `replayFailedStateSync()`. The `StateSender` is controlled by an owner that can register or deregister an authorized caller for a given receiver address.

- **Polygon → Ethereum (checkpoint commitments, execution via proofs).** In the opposite direction, Polygon PoS regularly submits checkpoints to Ethereum that commit to a range of Polygon blocks. These checkpoints act as trust anchors: to make Ethereum act on a Polygon event or state transition, the caller provides the relevant data together with an inclusion proof. Upon successful verification against the checkpoint commitment stored on Ethereum, the corresponding L1 contract executes the requested action. Checkpoints are submitted by Polygon's validator infrastructure roughly every 20-30 minutes.

2.2.2 sPOLController

The `sPOLController` manages the delegation of POL to Polygon validators on behalf of users. When POL is delegated to a validator, the controller receives `dPOL` (delegated POL) - a validator-specific ERC20 token representing the delegated stake. In return, the controller mints `sPOL` to the user, a liquid staking token representing their share of the total delegated position.

The controller is implemented as an upgradeable contract using OpenZeppelin's `Initializable`, `PausableUpgradeable`, `AccessManagedUpgradeable`, and `ReentrancyGuardTransient`. It is the core of the staking system and stores the system's delegated POL balance, validator configuration, pending reward fees, and user withdrawal queues.

2.2.2.1 Accounting

The controller tracks three key balances for exchange rate calculations. Note that the system assumes a 1:1 exchange rate between POL and dPOL at all times (refer to the Trust Model section for more details).

- `totaldPOLBalance`: the total amount of POL delegated across all validators; this value increases when users deposit POL or when staking rewards are restaked; includes the fee portion tracked in `feedPOLBalance`
- `totalsPOLBalance()`: returns the total supply of sPOL tokens (equivalent to `sPOLToken.totalSupply()`)
- `feedPOLBalance`: the portion of `totaldPOLBalance` allocated as protocol fees from restaking rewards; reserved for the fee receiver and excluded from exchange rate calculations

The effective backing for user-held sPOL is `totaldPOLBalance - feedPOLBalance`.

2.2.2.2 Validator Management

Validators in the sPOL staking system can be `INACTIVE`, `ACTIVE`, `DEACTIVATED`, or `FROZEN`. Only active validators can receive deposits or be selected for withdrawals. Frozen validators retain their stake but are excluded from new deposits and withdrawal selection. Unregistered validators default to `INACTIVE`; removed validators are set to `DEACTIVATED`. Except for `INACTIVE` validators, the system assumes a 1:1 exchange rate between POL and dPOL at all times for all validators (see Trust Model).

The controller contract exposes `restricted` validator management functions, callable only by addresses authorized by the `AccessManager` authority.

- `addValidator()`: registers a new validator by its validator ID, if it has not already been added and is verified to be active via the `StakeManager`; the new validator is stored in the `validators` mapping, with its `totalStaked` and `depositShare` amounts set to 0; its ID is appended to both arrays `validatorList` and `activeValidators`

- `removeValidator()`: sets an active validator with zero staked balance, no outstanding shares, and no pending rewards to `DEACTIVATED`; the validator is removed from `activeValidators` but remains in `validatorList`
- `freezeValidator()`: sets an active validator with zero deposit shares to `FROZEN`; the validator is removed from `activeValidators` but any existing stake remains delegated
- `unfreezeValidator()`: sets a frozen validator back to `ACTIVE` and re-adds its ID to `activeValidators`
- `updateValidatorTargetShare()`: updates the target allocation weights for stake distribution (`depositShare`) across a batch of active validators; enforces that weights across all active validators sum to exactly 100%
- `migrateValidator()`: transfers a specified amount of delegated stake from one validator to another via `StakeManager.migrateDelegation()`; optionally restakes pending rewards on both validators beforehand
- `reloadAllActiveValidatorInfo()`: re-syncs each active validator's `totalStaked` by reading share balances from their validator contracts, updating `totaldPOLBalance` accordingly
- `reloadAllValidatorInfo()`: same as above but includes all validators (including frozen ones, and deactivated ones)

2.2.2.3 Validator Restaking and Fee Management

Validator restaking is exposed via public functions and can be triggered by anyone calling `restakeValidator()` or `restakeAllActiveValidators()`. Restaking claims accrued staking rewards from a validator and compounds them back into the delegated position, increasing the validator's `totalStaked` and the system's `totaldPOLBalance`. A portion of the restaked rewards (determined by `rewardFee`) is retained as protocol fees and tracked in `feedPOLBalance`. Fees are only collected when rewards are restaked, not on user deposits or withdrawals.

Accumulated fees can be collected via the restricted `takeFee()` function, which converts `feedPOLBalance` to sPOL and mints the corresponding amount to the `feeReceiver`. The `rewardFee` percentage and `feeReceiver` address can be updated via `changeRewardFee()` and `changeFeeReceiver()` respectively, both of which are also restricted. Note that `changeFeeReceiver()` automatically triggers fee collection before updating the receiver. Based on the deployment configuration, the `feeReceiver` is initially set to the same address as the `AccessManager` authority.

2.2.2.4 Validator Selection

When users buy or sell `sPOL` without specifying a validator to delegate to, the system selects validators via the internal `_selectValidators()` function. This function aims to optimize the distribution of POL staked through the `sPOLController` among registered, active validators according to each validator's target `depositShare`, plus or minus the `maxDivergence` tolerance. For deposits, validators below their target share are prioritized; for withdrawals, validators above their target share are prioritized. If the requested amount cannot be satisfied within these constraints, the function falls back to distributing across all active validators (evenly for deposits, proportionally to available stake for withdrawals). This abstracts validator selection away from users, as the system distributes stake according to these predefined metrics.

2.2.2.5 Buying and Selling sPOL (on L1)

The controller contract exposes multiple external functions for buying and selling the staked POL token `sPOL`. `sPOL` is an upgradeable, non-rebasing ERC20 token with permit functionality (EIP-2612), deployed on Ethereum mainnet. Only the `sPOLController` can mint and burn `sPOL`.

Buying sPOL

When buying `sPOL` tokens, the general mechanism is as follows: POL is transferred from the user to the controller (requiring prior approval or a valid permit), which then delegates it to either a user-specified validator or one selected via `_selectValidators()`. Upon delegation, the controller receives `dPOL` (delegated POL) from the respective `ValidatorShare` contract. As noted in the Accounting section, the system assumes a 1:1 exchange rate between POL and `dPOL`. This relies on the fact that validator slashing is disabled for the validators that will be added to the controller, which is further discussed in the Trust Model section.

For the amount of POL deposited, a corresponding amount of `sPOL` is minted to the user based on the current POL-to-sPOL exchange rate. The following buy functions are exposed to users. Variants with `Permit` allow approvals via EIP-2612 signatures, and variants with a `validator` parameter allow users to specify a validator to stake with instead of relying on `_selectValidators()`:

- `buySPOL()` / `buySPOLPermit()`: transfers POL from the user, delegates to system-selected validators, and mints sPOL to the user
- `buySPOL(validator)` / `buySPOLPermit(validator)`: same flow, but delegates to a user-specified validator
- `buySPOLWithDPOL(validator)` / `buySPOLWithDPOLPermit(validator)`: allows users to buy sPOL using dPOL they already hold from a specified validator; if the validator is active and has enough capacity, the dPOL stays with that validator, otherwise the delegation is transferred to the controller and migrated to system-selected validators; sPOL is minted to the user based on the provided dPOL amount

Selling sPOL

When selling `sPOL` tokens, the general mechanism is as follows: the system burns the user's `sPOL` and initiates an unbonding request from either a user-specified validator or one selected via `_selectValidators()`. On the `ValidatorShare` contract, any pending rewards are restaked, and the corresponding dPOL shares are burned from the controller. The unbonding amount is added to the validator's withdraw pool. The controller reduces `totaldPOLBalance` and the validator's `totalStaked` accordingly.

Since undelegating requires a waiting period (enforced by the `StakeManager`), the POL is not immediately available. The user's pending withdrawal is added to a queue with a unique nonce. A user can have multiple withdrawal requests open at any time. Once the unbonding period has elapsed, the user can call `withdrawPOL()` to claim their POL.

The following sell and withdraw functions are exposed to users. As with buying, variants with `Permit` allow approvals via EIP-2612 signatures, and variants with a `validator` parameter allow users to specify a validator instead of relying on `_selectValidators()`:

- `sellSPOL()` / `sellSPOLPermit()`: burns sPOL from the user, initiates unbonding from system-selected validators, and adds the user to the withdrawal queue
- `sellSPOL(validator)` / `sellSPOLPermit(validator)`: same flow, but unbonds from a user-specified validator
- `withdrawPOL()`: claims POL for all matured withdrawal requests of the caller
- `withdrawPOL(user)`: same as above, but claims on behalf of a specified user

2.2.2.6 Administrative & View Functions

Moreover, the following restricted administrative functions are available:

- `cleanUpMaticPOL()`: in case there is any MATIC or POL dust in the controller contract, this function converts MATIC to POL (via the migration contract) and stakes the resulting POL balance to a specified validator; a fee is taken from that amount and added to `feedPOLBalance`.
- `pauseUserFunctions()` / `unpauseUserFunctions()`: pauses or unpauses the contract; pausing disables buying, selling, withdrawing, and restaking
- `changeMaxDivergence()`: updates the `maxDivergence` tolerance for validator selection; validators can deviate from their target `depositShare` by up to this many percentage points before being deprioritized

Additionally, the following public view functions are available:

- `totalsPOLBalance()`: returns the total supply of sPOL
- `convertPOLtoSPOL()`: calculates the sPOL amount for a given `_amountPOL` based on the current exchange rate; computed as:

```
_amountPOL * totalsPOLBalance() / (totaldPOLBalance - feedPOLBalance)
```

- `convertSPOLtoPOL()`: calculates the POL amount for a given `_amountSPOL` based on the current exchange rate; computed as:

```
_amountSPOL * (totaldPOLBalance - feedPOLBalance) / totalsPOLBalance()
```

- `getMostUnderfundedValidator()`: returns the validator furthest below its target share
- `getMostOverfundedValidator()`: returns the validator furthest above its target share

2.2.3 sPOLChild

The `sPOLChild` contract serves as both the L2 representation of the `sPOL` token (ERC20) and the cross-chain message tunnel on Polygon PoS. This allows users to buy and sell `sPOL` directly on the child chain. The contract performs local accounting for all user operations—tracking minted sPOL, burned sPOL, and withdrawal obligations—without immediate cross-chain communication.

The contract is implemented as an upgradeable contract using OpenZeppelin's `Initializable`, `PausableUpgradeable`, `AccessManagedUpgradeable`, `ERC20PermitUpgradeable`, and `ReentrancyGuardTransient`. It also inherits from `BaseChildTunnel` to receive and send messages on L2 and `MsgCoder` for message encoding and decoding.

2.2.3.1 Local Accounting

The contract tracks balances across two domains: local L2 state and the L1 sPOL and dPOL balances synced via cross-chain messages. Local accounting allows user operations to proceed without immediate cross-chain communication, while the synced L1 balances serve as the reference for exchange rate calculations (though they may become stale between updates).

- `polBalance`: total POL held by the `sPOLChild` contract
- `locallyMintedSPOL`: sPOL minted on L2 that is not yet backed by real stake on L1
- `locallyToBeBurnedSPOL`: sPOL sold on L2 awaiting burn settlement with L1
- `reservedWithdrawPOLBalance`: POL reserved and available for user withdrawals

- `requestedWithdrawPOLBalance`: POL requested from L1 via backfill, not yet arrived
- `missingWithdrawPOLBalance`: POL owed to users who sold sPOL but haven't withdrawn yet

2.2.3.2 Exchange Rate and Safety Fee

Assumption: The L1 exchange rate (dPOL/sPOL) is monotonically increasing, as validator rewards get restaked and slashing is disabled (see Trust Model).

Buying and selling sPOL on L2 uses the L1 token balances as reference for exchange rate calculations. The `sPOLController` on L1 holds the ground truth; the L2 contract stores synced values that may become outdated between updates:

- `l1SPOLBalance` / `l1DPOLBalance`: synced values from L1 representing the total sPOL supply and backing dPOL; updated via cross-chain messages from the `sPOLMessenger`
- `convertPOLtoSPOL()`: calculates the sPOL amount for a given `_amountPOL`, applying the `safetyFee`:

```
(_amountPOL * l1SPOLBalance * (SAFETY_FEE_DENOMINATOR - safetyFee)) / (l1DPOLBalance * SAFETY_FEE_DENOMINATOR)
```

- `convertSPOLtoPOL()`: calculates the POL amount for a given `_amountSPOL`:

```
(_amountSPOL * l1DPOLBalance) / l1SPOLBalance
```

The `safetyFee` should compensate for the delayed exchange rate updates on L2, keeping the buying rate for sPOL less favorable on L2 than on L1. It must be calibrated with respect to the expected validator reward, and various ways of inflating the price of sPOL on the L1, for the maximally allowed delay period (see Trust Model).

For selling, no fee is applied; the synced L1 rate is used directly. With the assumption that the L1 rate increases monotonically, selling conditions on L2 are at most as good as on L1.

Additionally, buying on L2 is disabled if the exchange rate hasn't been updated within `maxExchangeRateUpdateDelay` (default 30 days). Exchange rate updates from L1 are triggered manually by an authorized operator.

2.2.3.3 Buying and Selling sPOL (on L2)

- `buySPOL()`: user sends native POL (payable function); if the exchange rate is not outdated, the contract mints sPOL according to `convertPOLtoSPOL()` and transfers it to the user; the minted amount is tracked in `locallyMintedSPOL` for later settlement with L1
- `sellSPOL()`: user transfers sPOL to the contract; the corresponding POL amount is calculated via `convertSPOLtoPOL()` and tracked in `missingWithdrawPOLBalance`; the user's withdrawal is added to their queue, associated with the next `backFillCycle`
- `withdrawPOL()`: iterates through the user's outstanding withdrawal queue and transfers POL for all requests whose `backFillCycle` has been completed

Users can also bridge sPOL between L1 and L2, as explained in the Token Bridging section.

2.2.3.4 Cross-Chain Message Handling

The `sPOLChild` contract communicates with L1 via the `sPOLMessenger` through the following message types:

Outgoing messages (L2 → L1):

- `L2_MIGRATION_REQUEST`: triggers staking of bridged POL on L1 and minting of corresponding sPOL
- `L2_BACKFILL_REQUEST`: triggers unbonding of POL on L1 to cover pending withdrawals on L2

Incoming messages (L1 → L2):

- `EXCHANGE_UPDATE`: updates `l1SPOLBalance` and `l1DPOLBalance`, and triggers `_balanceWithL1()`; reverts if the new exchange rate is lower than the current one
- `L1_BACKFILL_RESPONSE`: confirms that requested POL has been bridged from L1; increases `reservedWithdrawPOLBalance` and marks the backfill cycle as complete

All other message types cause the contract to revert.

2.2.3.5 Reconciliation with L1

The `sPOLChild` contract reconciles local L2 operations with L1 through two mechanisms: *migration* (when surplus POL needs to be staked on L1) and *backfill* (when POL must be unstaked on L1 to cover withdrawals). Reconciliation can only be triggered when no migration or backfill is already ongoing.

When `_balanceWithL1()` is called, the contract evaluates its local state:

- **Migration**: if there is surplus POL and more sPOL has been minted locally than burned, the excess POL is bridged to L1 for staking; the corresponding sPOL is minted and locked on L1, backing the L2 representation
- **Backfill**: if the contract lacks sufficient POL to cover pending withdrawals, it sends a backfill request to L1; upon completion of the unbonding period, the POL is bridged back to L2 and made available for user withdrawals
- **Local balancing**: if surplus POL is sufficient to cover pending withdrawals without L1 involvement, the contract moves POL directly to `reservedWithdrawPOLBalance` and completes the backfill cycle immediately

2.2.3.6 Restricted Functions

The following functions are restricted, thus only callable by addresses authorized by the AccessManager:

- `changeSafetyFee()`: updates the safety fee applied when buying sPOL on L2 (maximum 1%)
- `setMaxExchangeRateUpdateDelay()`: updates the maximum validity window of the exchange rate before buys are disabled
- `pauseUserFunctions()` / `unpauseUserFunctions()`: pauses or unpauses user-facing functions (buying, selling, and withdrawing)
- `balanceWithL1()`: triggers reconciliation with L1 via migration or backfill

2.2.4 sPOLMessenger

The `sPOLMessenger` contract is the L1 counterpart tunnel to `sPOLChild`, handling cross-chain message processing and coordinating with the `sPOLController` to execute staking operations on behalf of L2 users. It serves as the bridge between L2 user activity and L1 stake management.

The contract is implemented as an upgradeable contract using OpenZeppelin's `Initializable`, `AccessManagedUpgradeable`, and `ReentrancyGuardTransient`. It inherits from `BaseRootTunnel` to receive and send messages on `L1` and `MsgCoder` for message encoding and decoding.

2.2.4.1 Cross-Chain Message Handling

The messenger receives messages from L2 that have been verified against checkpoints posted earlier. Upon receiving an `L2_MIGRATION_REQUEST`, it stakes the bridged POL via `sPOLController.completeMigration()` and bridges the minted sPOL back to L2. Upon receiving an `L2_BACKFILL_REQUEST`, it initiates unstaking via `sPOLController.startBackfillSell()` and records the backfill cycle for later completion.

2.2.4.2 Backfill Process

Backfills are a two-phase process due to the unbonding period required when unstaking from validators. In the first phase, triggered by an `L2_BACKFILL_REQUEST`, the messenger calls `sPOLController.startBackfillSell()` to initiate unbonding and stores the expected POL amount in `backfillAmounts`. Only one backfill can be active at a time, tracked via `currentActiveBackfillCycle`.

In the second phase, after the unbonding period on `L1` has elapsed, an authorized operator calls `completeBackfill()`. This function claims the unstaked POL from the controller, bridges it to L2 via the `DepositManager`, and sends an `L1_BACKFILL_RESPONSE` message to notify `sPOLChild` that the requested POL has arrived. The `completedBackfill` mapping tracks which cycles have been finalized.

2.2.4.3 Restricted Functions

- `updateL2ExchangeRate()`: reads the current exchange rate from the `sPOLController` and sends an `EXCHANGE_UPDATE` message to L2
- `completeBackfill()`: finalizes an active backfill after unbonding completes; withdraws POL and bridges it to L2

2.2.5 Token Bridging

The sPOL system bridges two tokens between Ethereum and Polygon PoS: POL and sPOL. Each token uses a different bridging infrastructure, and the `PolBridger` contract serves as a helper to coordinate POL transfers.

2.2.5.1 POL Bridging

In the context of this system, POL bridging is performed by the `sPOLMessenger` and `sPOLChild`, relying on the `PolBridger`, which is deployed on both chains at the same address. This is required for the L2 to L1 direction, as the bridge does not support operations to different addresses.

POL is bridged using the `DepositManager`, `WithdrawManager`, and `ERC20PredicateBurnOnly` contracts. Bridging POL from L2 to L1 works as follows:

1. On `L2`, the `sPOLChild` calls `PolBridger.bridgePOLToL1()`, with the corresponding `msg.value`, the latter calls `withdraw()` on the L2 POL token, which burns POL and emits an event

2. On L1, `PolBridger.exitPOL()` must be called with a proof of the burn event, initiating the exit via `ERC20PredicateBurnOnly.startExitWithBurntTokens()`
3. After the challenge period, `PolBridger.finalizeExitPOL()` calls `WithdrawManager.processExits()` to release the POL to the `PolBridger` through the `DepositManager`
4. The `sPOLMessenger` can then call `PolBridger.takePOLL1()` to retrieve the POL

Note that step 2 and 3 can be performed by anyone.

Bridging POL from L1 to L2 is simpler:

1. The `sPOLMessenger` calls `DepositManager.depositERC20ForUser()` and deposits for the `sPOLChild`
2. Given the event emitted, the system address on L2 calls the `StateVerifier`, which in turn calls `ChildChain.onStateReceive()`
3. The `ChildChain` contract is the owner of the L2 POL token. It uses `POL.deposit()` to send POL to the `sPOLChild`.

2.2.5.2 sPOL Bridging

sPOL is bridged using the `RootChainManager`, `ChildChainManager`, and `ERC20Predicate` contracts. Bridging sPOL from L1 to L2 works as follows:

1. On L1, `RootChainManager.depositFor()` locks sPOL in the `ERC20Predicate`
2. On L2, the `StateSyncer`, called by the system address, calls `childChain.onStateReceive()`
3. The `ChildChainManager` calls `sPOLChild.deposit()`, which mints the corresponding sPOL to the recipient

Bridging sPOL from L2 to L1:

1. On L2, `sPOLChild.withdraw()` burns the user's sPOL
2. The user proves the burn event with `RootChainManager.exit()` to unlock sPOL from the `ERC20Predicate` on L1

2.2.5.3 Migration and Backfill Bridging

During migration, POL is bridged from L2 to L1 via the `PolBridger`, and sPOL is bridged back from L1 to L2 via the `RootChainManager`. The `sPOLChild.deposit()` function recognizes incoming sPOL from an ongoing migration and completes the migration without minting additional tokens.

During backfill, sPOL is burned on L2 (via `_burnSPOLForMessenger()`), and after unbonding completes on L1, POL is bridged back to L2 via the `DepositManager`.

2.2.5.4 Direct sPOL Bridging

Users can bridge sPOL independently of the staking system:

- **L1 → L2:** User deposits sPOL via the `RootChainManager` on L1, which locks the tokens and triggers a state sync; on L2, the `ChildChainManager` receives the state sync and calls `sPOLChild.deposit()` to mint sPOL to the user
- **L2 → L1:** User calls `sPOLChild.withdraw()` to burn sPOL on L2, then submits a proof of the burn event on L1 to unlock sPOL from the predicate

2.2.6 Changes in Version 2

The most significant change in Version 2 is how migrations and backfills are processed on L1. In Version 1, the `sPOLMessenger` called dedicated functions on the `sPOLController` (`completeMigration()` and `startBackfillSell()`) that operated with token amounts determined by the L2 rate. In Version 2, the `sPOLMessenger` is no longer registered with the `sPOLController` and instead interacts with it as a regular user, calling `buySPOL()` and `sellSPOL()` directly. As a result, migrations and backfills execute at the current L1 exchange rate. Any excess POL, arising from the difference between what L2 bridged and what is required at the L1 rate, is transferred to the `sPOLController` as unstaked balance, to be staked when the admin calls `cleanUpMaticPOL()`.

On the `sPOLController` side, several validator management changes were introduced. `reloadAllValidatorInfo()` has been replaced by `reloadValidatorInfo()`, allowing the admin to re-sync a single validator's stake rather than iterating over all validators. A `MAX_DIVERGENCE` constant now bounds the allowed staking divergence per validator. The validator freeze mechanism (`FROZEN` status, `freezeValidator()`, `unfreezeValidator()`) has been removed entirely. Validator selection via `_selectValidators()` now queries `ValidatorShare.locked()` and skips locked validators for deposits. Zero-amount buy and sell operations are no longer permitted.

On L2, the default `maxExchangeRateUpdateDelay` has been reduced from 30 to 10 days. Several cross-chain message handling paths in `sPOLChild` that previously reverted now return silently with event emission instead: an exchange rate update that would decrease the rate emits `ExchangeRateDeclined` and preserves the previous (higher) rate, and an update arriving while a migration or backfill is already ongoing emits `BalancingAlreadyOngoing` and defers reconciliation. Since these messages no longer revert, they cannot be replayed permissionlessly until they succeed but are dropped instead. Additionally, `sellSPOL()` now enforces a freshness check on the exchange rate, a restriction that previously applied only to buys on L2. The safety fee can no longer be set to zero, and unpausing buy and sell operations requires a fresh exchange rate.

Moreover, the `PolBridger` now includes a `rescueNative()` function for recovering native tokens.

Overall, Version 2 introduces reentrancy guards on all user-facing functions, NatSpec documentation, various gas optimizations, and general code cleanup.

2.2.7 Changes in Version 3

In the `PolBridger`, the POL token address used in `finalizeExitPOL()` has been replaced with the MATIC token address on L1. This is necessary because the `WithdrawManager` and `ERC20PredicateBurnOnly` still operate on the MATIC token address. The L2 `MRC20` token emits `Withdraw` events referencing the MATIC address, which must match the token address used when calling `processExits()` on L1.

In the `sPOLController`, the NatSpec on `reloadValidatorInfo()` has been corrected.

2.2.8 Changes in Version 4

Only one change to a minor issue was introduced in the version 4 of the system.

2.3 Trust Model

2.3.1 Fundamental System Assumptions

2.3.1.1 Solidity Version and EVM Version

All contracts in the system are compiled using Solidity version 0.8.30, targeting the `Prague` EVM version.

2.3.1.2 Protocol Invariants

- **dPOL:POL rate is always 1:1** - Validator shares (dPOL) are assumed to always be redeemable 1:1 for POL. This should hold for every validator in the system, even if they are deactivated or inactive. This implies that no slashing occurs. In practice this means that:
    - `ValidatorShare.exchangeRate()` always returns 1e29
    - `ValidatorShare.withdrawExchangeRate()` always returns 1e29
- **sPOL:POL exchange rate is monotonically increasing** - The exchange rate between sPOL and POL is assumed to never decrease. This relies on the fact that staking rewards are restaked regularly, and no slashing occurs. This implies that users never lose value by holding sPOL instead of POL.

2.3.1.3 Initial supply of sPOL

In this report, we assume the Polygon team performs a one-time initial buy of sPOL in the `sPOLController` immediately after deployment and before any other user interacts with the contract. This seeding is essential to the security of the system. The initial supply must:

- Be large enough to make any first depositor inflation attack against reasonably sized deposits require capital that is not possible to acquire.
- Be permanently locked to avoid re-initialization attacks.
- Not be bridged to L2, or an attacker could mint the corresponding amount on L2 and bridge it back to L1 to drain the controller's initial liquidity.
- Be minted at a 1:1 exchange rate (1 POL deposited mints 1 sPOL).

2.3.1.4 Exchange Rate Safety

- **``safetyFee`` >= expected exchange rate change during ``maxExchangeRateUpdateDelay``**: If this does not hold, sPOL on L2 becomes more attractive than on L1, creating arbitrage opportunities.
- **Default configuration**: `maxExchangeRateUpdateDelay = 10 days`, `safetyFee = 0.3%` (expecting ~0.25% rewards per month)
- It is assumed that the exchange rate on L1 will be bridged to L2 often, to avoid stale rates.

2.3.2 Roles & Trust Levels

2.3.2.1 Upgrade Authority (Proxy Admin)

Trust Level: FULLY TRUSTED

This role can change the implementations of the upgradeable contracts and thereby change all invariants and controls.

Capabilities

- Upgrade implementations of `sPOLController`, `sPOLChild`, `sPOLMessenger`, and `sPOL`

Trust Requirements

- Must preserve storage layout and critical invariants across upgrades

Worst-Case Consequences if Malicious/Compromised

- Full control of funds and protocol logic, including disabling withdrawals and stealing user funds

2.3.2.2 AccessManager Authority

Trust Level: FULLY TRUSTED

Capabilities

- Authorize and deauthorize callers for restricted functions in all contracts of the system.
- Transfer the Authority role to a new AccessManager.

Worst-Case Consequences if Malicious/Compromised

- Full control of all restricted functions, leading to the same consequences as a malicious Restricted Role

2.3.2.3 Authorized Callers of Restricted Functions

Trust Level: FULLY TRUSTED

The Authority is expected to authorize trusted entities to call certain restricted functions. This authorization is given based on the caller's address, the function selector, the contract address, and might be time-dependent. In the context of this system, we assume that only a single entity is authorized to call all restricted functions of all contracts. We refer to that entity as the owner, or the admin.

Capabilities

- **sPOLController**: `addValidator`, `removeValidator`, `updateValidatorTargetShare`, `migrateValidator`, `reloadAllActiveValidatorInfo`, `reloadValidatorInfo`, `changeMaxDivergence`, `changeFeeReceiver`, `changeRewardFee`, `takeFee`, `pauseUserFunctions`, `unpauseUserFunctions`, `cleanUpMaticPOL`
- **sPOLChild**: `balanceWithL1`, `changeSafetyFee`, `setMaxExchangeRateUpdateDelay`, `pauseBuySell`, `unpauseBuySell`
- **sPOLMessenger**: `completeBackfill`, `updateL2ExchangeRate`
- **PolBridger**: `initialize`, `rescue`, `rescueNative`, `pause`, `unpause`

Trust Requirements

- Must execute actions in the correct order when multiple steps are required
- Must thoroughly check system state before performing restricted actions
- Must provide enough gas for operations like `completeBackfill` to prevent OOG reverts
- Must trigger backfills regularly so users who sell sPOL can eventually withdraw

Worst-Case Consequences if Malicious/Compromised

- Could pause all user functions indefinitely
- Could manipulate fee collection and redirect fees
- Could manipulate validator stakes in ways that affect exchange rates
- Could prevent backfills from completing, locking user withdrawals on L2
- Could set `maxExchangeRateUpdateDelay` too long, allowing stale exchange rates
- Could set `safetyFee` too low, enabling L1/L2 arbitrage
- Could drain the `polBridger` of its balance using the `rescueNative` function

2.3.2.4 Fee Receiver

Trust Level: UNTRUSTED (but designated)

- Receives accumulated protocol fees as sPOL

- Cannot affect protocol operations
- Only receives tokens, cannot pull or manipulate state

2.3.2.5 Regular Users

Trust Level: UNTRUSTED

Capabilities

- All external but not restricted functions provided by the protocol

2.3.3 External Protocol Dependencies

The following external contracts are used by the sPOL staking system. Their correct operation is assumed as part of the trust model. More specific trust assumptions for each contract follow.

2.3.3.1 StakeManager (Polygon Staking Contract)

- All `validatorShare` contracts are owned by the `StakeManager`. The `StakeManager` is trusted. The governance of the `StakeManager` is trusted. The stakers of the validators added to the sPOL system are trusted.
- `updateValidatorContractAddress()` must not be used on validators added to the sPOL system.
- Validators used by sPOL should generally have `deactivationEpoch` set to 0
- No foundation validator is used by sPOL (`validatorID >= 8`).
- The functions of the `StakeManager` used by the sPOL system operate correctly.
- `delegationEnabled` is assumed to always be true

2.3.3.2 ValidatorShare Contracts

- All `ValidatorShare` contracts used by the sPOL system operate correctly.
- Validators cannot be slashed.
- `exchangeRate()` and `withdrawExchangeRate()` always return 1e29

2.3.3.3 Polygon Migration Contract

- `migrate()` correctly converts MATIC to POL at 1:1 rate

2.3.3.4 StateSender / StateSyncer

- L1↔L2 message finality assumptions: reorgs and delayed finality are handled appropriately
- Messages are assumed to be delivered correctly and without tampering (eventual delivery; retries may be required)
- Messages can be delivered out of order
- Failed message deliveries may be retried on L2 permissionlessly
- Replay protection is implemented
- The `sPOLChild` is added as a valid receiver on L2 for the `sPOLMessenger` on L1
- System messages emit events that are included in block logs, meaning proofs of these event emissions can be verified on L1.

2.3.3.5 RootChainManager / ChildChainManager / ERC20Predicate (sPOL Bridge)

- Handles sPOL token bridging between L1 and L2
- Bridge correctly processes deposits and withdrawals
- sPOL was correctly mapped in all contracts to ensure proper bridging, including the predicate for sPOL being registered in the `RootChainManager`, and the root and child token mapped in both the L1 and the L2.
- Inherits the trust assumptions of the underlying StateSender/StateSyncer mechanism
- All contracts involved in the sPOL Bridge, for which addresses are hardcoded in the sPOL system, are assumed to never change. For example, the predicate contract for sPOL is assumed to always be the same in the future, and not be migrated to a new one.
- The different trusted roles of the `RootChainManager`, `ChildChainManager`, and `ERC20Predicate` are assumed to be operated correctly to avoid changing critical parameters in a way that would break the bridging functionality for sPOL.
- Nothing should prevent bridging 0 sPOL tokens, as this might happen during migration/backfill flows.

2.3.3.6 DepositManager / WithdrawManager / ERC20PredicateBurnOnly (POL Bridge)

- Handles POL bridging
- Bridge correctly processes deposits and withdrawals
- Inherits the trust assumptions of the underlying StateSender/StateSyncer mechanism
- `maxErc20Deposit` for POL is assumed to be sufficiently high to handle all deposits, including the ones triggered by backfills (At the time of writing, set `1_000_000_000_000` POL)
- When performing a POL migration, the admin should ensure that the L2 transaction emits at most 10 events. Failing to do so could cause the bridge transaction to be impossible to relay on L1 due to a check in the `ERC20PredicateBurnOnly`.
- All contracts involved in the bridge, for which addresses are hardcoded in the sPOL system, are assumed to never change. For example, the predicate contract for POL is assumed to always be the same in the future.
- The different trusted roles of the `DepositManager`, `WithdrawManager`, and `ERC20PredicateBurnOnly` are assumed to be operated correctly to avoid changing critical parameters in a way that would break the bridging functionality for sPOL.
- The root token address used by the L2 `MRC20` and `childChain` contracts is assumed to be `MATIC`. On L1, for messages to L2, the `DepositManager` rewrites POL messages as MATIC; for messages from L2, the `DepositManager` translates MATIC back to POL. This is required because the L2 side of the bridge still operates with the MATIC address, and events emitted by the L2 MRC20 token reference that MATIC address.

2.3.4 Contract-Specific Trust Assumptions

2.3.4.1 sPOLController (L1)

- **Active validators list should be reasonably small** given transaction gas limits (~16M gas budget) such that all operations iterating over active validators can complete
- **Validators list** should also not grow too large to avoid hitting gas limits in functions iterating over all validators

- **Restaking should be done manually before buys/sells/bridging the exchange rate to L2** to ensure correct exchange rate (not enforced, using stale rate otherwise)
- **The controller should never be reset**: this would enable first depositor inflation attacks, and the L2 exchange rate assumptions would break as it could then be greater than on L1.
- **The weight distribution across validators is only a soft constraint**; the algorithm to pick validators may deviate from target shares and will not guarantee exact allocations.
- **RewardFee should not be set to 100%**: this could cause divisions by 0 when calculating exchange rates.
- **Validators should not be reloaded when the total supply of sPOL is zero**: This would cause a large increase in exchange rate, after the first sPOL buy.

2.3.4.2 sPOLChild (L2)

- **Exchange rate should not be bridged from L1 while the Controller is empty**: To avoid division by zero and invalid exchange rates on L2
- **Exchange rate can only increase** - relies on no slashing and no reinitialization of the controller
- **Safety fee must be calibrated correctly** to prevent L1/L2 arbitrage
- **Exchange rate updates must be triggered regularly** to prevent buy and sell operations from being blocked
- **Admin should regularly trigger backfills** so users assigned to "next cycle" can eventually withdraw

2.3.4.3 PolBridger

- **Both L1 and L2 contracts must be deployed at the same address**: required by the Plasma bridge design

3 Limitations and use of report

Security assessments cannot uncover all existing vulnerabilities; even an assessment in which no vulnerabilities are found is not a guarantee of a secure system. However, code assessments enable the discovery of vulnerabilities that were overlooked during development and areas where additional security measures are necessary. In most cases, applications are either fully protected against a certain type of attack, or they are completely unprotected against it. Some of the issues may affect the entire application, while some lack protection only in certain areas. This is why we carry out a source code assessment aimed at determining all locations that need to be fixed. Within the customer-determined time frame, ChainSecurity has performed an assessment in order to discover as many vulnerabilities as possible.

The focus of our assessment was limited to the code parts defined in the engagement letter. We assessed whether the project follows the provided specifications. These assessments are based on the provided threat model and trust assumptions. We draw attention to the fact that due to inherent limitations in any software development process and software product, an inherent risk exists that even major failures or malfunctions can remain undetected. Further uncertainties exist in any software product or application used during the development, which itself cannot be free from any error or failures. These preconditions can have an impact on the system's code and/or functions and/or operation. We did not assess the underlying third-party infrastructure which adds further inherent risks as we rely on the correct execution of the included third-party technology stack itself. Report readers should also take into account that over the life cycle of any software, changes to the product itself or to the environment in which it is operated can have an impact leading to operational behaviors other than those initially determined in the business specification.

4 Terminology

For the purpose of this assessment, we adopt the following terminology. To classify the severity of our findings, we determine the likelihood and impact (according to the CVSS risk rating methodology).

- *Likelihood* represents the likelihood of a finding to be triggered or exploited in practice
- *Impact* specifies the technical and business-related consequences of a finding
- *Severity* is derived based on the likelihood and the impact

We categorize the findings into four distinct categories, depending on their severity. These severities are derived from the likelihood and the impact using the following table, following a standard risk assessment procedure.

Likelihood	Impact		
	High	Medium	Low
High	Critical	High	Medium
Medium	High	Medium	Low
Low	Medium	Low	Low

As seen in the table above, findings that have both a high likelihood and a high impact are classified as critical. Intuitively, such findings are likely to be triggered and cause significant disruption. Overall, the severity correlates with the associated risk. However, every finding's risk should always be closely checked, regardless of severity.

5 Open Findings

In this section, we describe any open findings. Findings that have been resolved have been moved to the Resolved Findings section. The findings are split into these different categories:

- Security: Related to vulnerabilities that could be exploited by malicious actors
- Design: Architectural shortcomings and design inefficiencies
- Correctness: Mismatches between specification and implementation

Below we provide a numerical overview of the identified findings, split up by their severity.

Critical-Severity Findings	0
High-Severity Findings	0
Medium-Severity Findings	1

- Locked or Disabled Validators Break sPOL Flows Code Partially Corrected

Low-Severity Findings	4

- Delayed Rewards Accounting Risk Accepted
- _balanceWithL1 Revert Risk Accepted
- Bridge Liquidity on L1 Risk Accepted
- Buy/Sell Rate Computed Without Restaking Rewards Risk Accepted

5.1 Locked or Disabled Validators Break sPOL Flows

Design Medium Version 1 Code Partially Corrected

CS-POLYGON-SPOL-005

In `ValidatorShare`, `_buyShares` reverts under either of these conditions:

- The validator is locked (`onlyWhenUnlocked` modifier).
- Delegation is disabled (`delegation` boolean check).

The lock can be set while `unstakePOL` or `unstake` is executing in `StakeManager`. Delegation can be disabled via `updateValidatorDelegation` in `StakeManager`.

Because several validator functions call `_buyShares` internally, they inherit this revert behavior:

- `restakeAndUnstakePOL` and `restakePOL` revert when liquid rewards exist and the validator is locked or delegation is disabled.
- `restakeAndTransferFrom` reverts when liquid rewards exist and delegation is disabled. If locked, it pays rewards out and continues (no restake).
- `restakeAndStakePOL` always reverts when locked or delegation is disabled.

Implications in `sPOLController`:

- `buySPOL` and `buySPOL(validator)` always revert if a selected validator is locked or delegation is disabled (they call `restakeAndStakePOL`).

- Sell paths revert when liquid rewards exist on selected validators (they call `restakeAndUnstakePOL`). There is no skip-restake sell path, so users can become stuck.

One possible recovery is for the admin to migrate the validator (with `_restake=false`) to a new validator that is not locked or delegation-disabled.

Additionally, if a validator has initiated unstaking but is still in its notice period (`deactivationEpoch > currentEpoch`), buy and sell operations will fail because `StakeManager.updateValidatorState()` reverts with "unstaking". If such a validator is selected, user operations revert, causing a practical DoS for those flows.

Code partially corrected:

- Multi-validator buy paths now skip locked validators instead of reverting.
- Single-validator buy paths still revert when the selected validator is locked or delegation-disabled, because the user explicitly chose that validator. This also applies to `_buySPOLWithDPOLMulti` when it takes the first branch (`restakeAndTransferFrom`) and attempts to restake.
- Sell paths now attempt restaking first; if restaking fails, they catch the error and continue with the sell without restaking. This avoids sell-path DoS in locked or delegation-disabled cases. Note in that case that rewards are sent to the controller directly. To restake them, `cleanUpMaticPOL` can be called by the admin.

Remaining gaps:

- When delegation is disabled for a validator, multi-validator buy paths will revert if that validator is selected, since the newly added skip logic only handles locked validators:

```
// Skip locked validators when buying
if (_buy && validator.validatorContract.locked()) {
    continue;
}
```

- When selling, even the no-restake fallback can revert while a validator is in its unstaking notice period, because the fallback still updates validator state. If such a validator is selected, the sell transaction cannot skip it and may still DoS sell operations.

For the remaining gaps, Polygon answered:

```
1. Delegation-disabled not skipped in buy paths: Disabling delegation has never
occurred on Polygon PoS; the established validator process for discouraging
delegation is setting commission to 100% instead. Adding a delegation check on
every buy results in a disproportionate gas cost.

2. Sell fallback reverts during unstaking notice period: DoS lasts at most 1
checkpoint (~30 min, possibly 1h). With few validators this is rare.
`sellSPOL(amount, validatorId)` with a valid target still works as a guaranteed
path for urgent sells. Admin can migrate the validator (`_restake=false`) if needed.
```

5.2 Delayed Rewards Accounting

Design | Low | Version 2 | Risk Accepted

CS-POLYGON-SPOL-043

In the Controller, the functions that buy sPOL using DPOL directly allow users to provide validator shares when purchasing sPOL. If the incoming validator share is either:

- From a validator that is not active in the sPOL system
- From a validator that is active but already over-invested

The system calls:

```
bool success = IValidatorShare(stakeManager.getValidatorContract(_validatorOfDPOL))
            .transferFrom(_user, address(this), _amount);
```

This function does not restake the Controller's accrued rewards and instead transfers them as POL to the Controller. This allows anyone to delay restaking for an over-invested active validator by buying sPOL with a small amount of shares from that validator. The admin will need to call `cleanUpMaticPOL` to manually restake those rewards.

Similar behavior may also occur in `_sellSharesFromValidator` when the fallback path is triggered, even though restaking could have succeeded. This could happen if the function is called with intentionally low gas, causing the `try/catch` path to fail while still leaving enough gas to execute the fallback. Because the fallback itself is gas-intensive, exploiting this behavior is likely impossible in practice.

Risk accepted:

Polygon accepted this risk and answered:

```
Rewards land safely in the Controller (not lost). `cleanUpMaticPOL`
exists to sweep them into staking. Frequent protocol usage auto-restakes
validators, so accrued rewards between restakes are small. No economic
incentive for an attacker.
```

5.3 `_balanceWithL1` Revert

Design | Low | Version 2 | Risk Accepted

CS-POLYGON-SPOL-044

In `sPOLChild`, `_balanceWithL1` may trigger a migration to L1 depending on system state. During migration, the system calls `polBridger.bridgePOLToL1`, which then calls `MRC20.withdraw` and emits an event consumed by the POL bridge.

Per the trust model, the POL bridge supports at most 10 relevant log emissions in a transaction that initiates bridging to L1. If more than 10 logs are emitted, the bridge transaction may become non-relayable on L1 due to the following check in `ERC20PredicateBurnOnly`:

```
require(logIndex < MAX_LOGS, "Supporting a max of 10 logs");
```

`_balanceWithL1` can be reached via:

1. `balanceWithL1()`: callable only by the admin.
2. `onStateReceive()` with message type `EXCHANGE_UPDATE`: callable by the StateSyncer.

In the first case, the admin should ensure that any transaction including `balanceWithL1()` stays below the 10-log limit. This should account for logs emitted by external dependencies (for example `authority.consumeScheduledOp`) and for future upgrades that may add new emissions. This also implies that the admin should not be any sort of Safe wallet or other multi-sig/governance contract that

emits logs on execution, or, most importantly that allows permissionless execution of calls once the threshold is met.

The second case has two sub-cases:

1. The system address calls `StateReceiver.commitState` and state-sync execution succeeds. As in the first case, the transaction must remain under the 10-log limit, including logs emitted by `StateReceiver` itself and any additional logs introduced by future upgrades.
2. The system address calls `StateReceiver.commitState`, but `sPOLChild.onStateReceive` fails. In that case, the message is stored in `StateReceiver` and can be retried **permissionlessly** via `StateReceiver.replayFailedStateSync` (or `replayHistoricFailedStateSync` if `rootSetter` calls `setRootAndLeafCount`). A caller could wrap this retry in a transaction that emits additional logs, attempting to make the migration non-relayable.

For this scenario to matter, `sPOLChild.onStateReceive` must fail at time A and later become executable at time B after state changes on the child chain. The only plausible cause identified is that `polBridger` is paused at time A and unpaused at time B. Future upgrades should be reviewed carefully to ensure they do not introduce additional failure modes that make replay success possible at a later time.

Risk accepted:

Polygon accepted this risk and answered:

```
Extremely unlikely — the only identified path (polBridger pause/unpause) must
occur within the ~30 min window before the rate update arrives via state sync.
Even if triggered, it's only a DoS, which we can resolve. Will re-evaluate if
more realistic paths are identified.
```

5.4 Bridge Liquidity on L1

Design | Low | Version 1 | Risk Accepted

CS-POLYGON-SPOL-006

During backfills, the system assumes the sPOL bridge has enough liquidity locked on L1 to cover the backfill amount.

If the sPOL bridge lacks liquidity on L1, an ongoing backfill cannot complete until the bridge is replenished.

For example, an attacker could do the following (even if the economic incentive is unclear):

- Wait for a backfill to occur on L2
- Deposit a large amount of POL into `sPOLChild`
- Bridge the obtained sPOL to L1 using `sPOLChild.withdraw`
- Call `rootChainManager.exit` for that operation on L1 before the backfill is processed

As a result, the `erc20Predicate` could temporarily lack sufficient funds to cover the backfill.

Until the bridge is replenished, messages between L1 and L2 will not be processed due to the backfill being stuck. This includes exchange rate updates. Note that this could make issue L1 Exchange-Rate Appreciation Can Brick L2 Migrations or Enable Arbitrage more likely to occur once the backfill completes.

Note that if the following invariant does not hold for the POL token, a similar issue can arise for migrations with the POL bridge:

```
POL balance of the POL Plasma bridge on L1 >= total supply of POL on L2
```

Risk accepted:

Polygon makes no changes to the codebase and states the following:

```
Design limitation inherent to the bridge architecture. No code fix possible.
```

5.5 Buy/Sell Rate Computed Without Restaking Rewards

Correctness | Low | Version 1 | Risk Accepted

CS-POLYGON-SPOL-007

When users buy or sell sPOL, the exchange rate calculation does not account for pending validator rewards that have accumulated since the last restake. For buys, sPOL is minted before restaking occurs, allowing buyers to immediately benefit from rewards they did not contribute to. For sells, sPOL is converted to POL before restaking, causing sellers to forfeit rewards they were entitled to.

The exchange rate is computed via `convertPOLtoSPOL()` and `convertSPOLtoPOL()`, which use `totaldPOLBalance`. However, `totaldPOLBalance` only reflects rewards that have been explicitly compounded via `restakeValidator()`. The buy and sell functions compute the exchange rate first, then perform restaking as a side effect of the validator operations afterward:

```
function _buySPOLSingle(uint256 _amount, uint16 _validator, address _user) internal returns (uint256) {
    uint256 toMint = convertPOLtoSPOL(_amount);
    // ...
    _buySharesFromValidator(validator, _amount);  // restaking happens here, after rate is determined
    _mintSPOL(_user, _amount, toMint);  // minted at stale rate
    // ...
}
```

The same pattern applies to sell operations.

Impact on Buyers

When buying sPOL, the amount minted is calculated at the stale rate. Once restaking occurs (either within the same transaction via `_buySharesFromValidator` or triggered externally), the buyer's sPOL immediately appreciates from rewards they did not contribute to. This effectively transfers value from existing sPOL holders to new buyers.

Impact on Sellers

When selling sPOL, the POL amount is calculated at the stale rate before restaking. The seller receives less POL than they would be entitled to if pending rewards were compounded first. These forfeited rewards are distributed to remaining sPOL holders.

Front-running Scenario

If rewards have accumulated since the last restake, an attacker observing a transaction that would trigger restaking of one or more validators (e.g. `restakeAllActiveValidators()`) can front-run it with a buy operation. The attacker purchases sPOL at the stale rate, and once the restake executes, their position immediately appreciates. Profitability depends on the time elapsed since the last restake; with

frequent restaking, the arbitrage window remains small. Given the withdrawal delays, such opportunities may be limited due to the inability to use flash loans and the need to lock funds for a period.

Risk accepted:

Polygon accepts this behavior as an intentional design choice:

```
Intentional design. Impact is small with frequent restaking.
Sell withdrawals require unbonding (no flash loan arbitrage possible).
```

6 Resolved Findings

Here, we list findings that have been resolved during the course of the engagement. Their categories are explained in the Open Findings section.

Below we provide a numerical overview of the identified findings, split up by their severity.

Critical-Severity Findings	0
High-Severity Findings	4

- Fund Lockup Due to Inconsistent Validation in sellSPOL and withdrawPOL Code Corrected
- L1 Exchange-Rate Appreciation Can Brick L2 Migrations or Enable Arbitrage Code Corrected
- Missing polBalance Decrease During Migration Code Corrected
- sPOLChild Missing Fallback to Receive POL Code Corrected

Medium-Severity Findings	0
Low-Severity Findings	8

- First Depositor Inflation Attack Specification Changed
- Incomplete Validator Status Checks on migrateValidator Specification Changed
- Incorrect Value Logged in BackfillCompleted Event Code Corrected
- Missing depositShare Validation on Validator Removal Code Corrected
- No Minimum Safety Fee Validation Code Corrected
- reloadAllValidatorInfo Reloads Deactivated Validators Code Corrected
- removeValidator Can Be Temporarily DoSed Code Corrected
- sPOLBurned Event Logs Incorrect Amount In Multi-Validator Sell Code Corrected

Informational Findings	19

- Division-by-zero in Redeem Distance Math Code Corrected
- Exchange-rate Decrease Code Corrected
- Function Visibility Too Broad Code Corrected
- Gas Savings Code Corrected
- Inconsistent Reentrancy Protection Code Corrected
- Incorrect Interface Code Corrected
- Missing Comprehensive NatSpec Documentation Code Corrected
- Missing or Superfluous Events Code Corrected
- Non-indexed Event Parameters Code Corrected
- Outdated or Pending Comments Code Corrected
- Solidity Version Not Fixed Code Corrected
- Typo in Variable Names Code Corrected
- Unused Errors Code Corrected
- Unused Imports Code Corrected
- Unused Variable spacer in sPOLController Code Corrected

- Use of State Variable Instead of Function Parameter Code Corrected
- depositData Encoding Code Corrected
- rescue Cannot Transfer POL Tokens Code Corrected
- rescue Incompatible With Non-Compliant ERC20 Tokens Code Corrected

6.1 Fund Lockup Due to Inconsistent Validation in `sellSPOL` and `withdrawPOL`

Design High Version 1 Code Corrected

CS-POLYGON-SPOL-001

In `sPOLController`, selling sPOL for POL is a two-step process:

1. `sellSPOL` is called. This function initiates the sell by invoking `restakeAndUnstakePOL` on the relevant `ValidatorShare` contract(s).
2. The user then calls `withdrawPOL` after the unstaking period has elapsed.

A validation mismatch between the two steps can lead to permanent fund lockup.

During sell operations, `ValidatorShare._sellVoucher` is invoked. This function allows selling 0 shares:

```
require(totalStaked != 0 && totalStaked >= claimAmount, "Too much requested");
```

In contrast, the withdrawal process calls `validatorShare._unstakeClaimTokens`, which explicitly requires a non-zero share balance to proceed:

```
require(
    unbond.withdrawEpoch.add(stakeManager.withdrawalDelay()) <= stakeManager.epoch() && shares > 0,
    "Incomplete withdrawal period"
);
```

- The validator selection in `sellSPOL` can result in a case where 0 shares are sold from one of the chosen validators, creating a pending withdrawal with 0 shares.
- More directly, if a user calls `sellSPOL(0)`, a pending withdrawal with 0 shares is created.

Because `sPOLController` enforces ordered processing of pending withdrawals per user (with no ability to skip a nonce), a single withdrawal with zero shares blocks all subsequent withdrawals indefinitely, leading to permanent fund lockup for the affected user.

Code corrected:

The `_sellSPOLMulti` and `_sellSPOLSingle` functions were updated to include checks that prevent selling 0 shares, ensuring that no pending withdrawal with 0 shares can be created.

6.2 L1 Exchange-Rate Appreciation Can Brick L2 Migrations or Enable Arbitrage

Design High Version 1 Code Corrected

CS-POLYGON-SPOL-002

General Design:

The exchange rate on L2 is always lagging behind the exchange rate (dPOL/sPOL) on L1. While this is unfavorable (and thus acceptable) for users selling sPOL on L2, it can be advantageous for users buying sPOL on L2 and lead to arbitrage. To mitigate this, a safety fee is applied when minting sPOL on L2 based on the delayed L1 rate. This safety fee was chosen to be large enough to cover roughly one month of expected rewards on L1 (which would increase the L1 exchange rate).

Other Ways for the L1 Exchange Rate to Appreciate:

Beyond normal rewards, the L1 exchange rate can also appreciate unexpectedly due to the following actions:

1. When the admin reloads validators, suddenly accounting for new dPOL with no corresponding sPOL minted (validators can receive donations).
2. When the admin calls `cleanUpMaticPOL`, inflating the POL balance used for exchange-rate calculations.
3. Via stealth donations (rounding directions when buying and selling sPOL in the controller) that inflate the POL balance on L1.
4. When completing L2 to L1 migrations:
    - The sPOL buys happen with an exchange rate inflated by the safety fee.
    - Completing the migration on L1 with `completeMigration` deposits more POL than what is needed to mint the required sPOL.
    - Due to the safety fee, the conversion functions on L2 are not inverses of each other. Normal usage (users buying and selling on L2) accumulates additional POL in sPOLChild, which further inflates the L1 rate when migrated. Repeated buy/sell cycles can amplify this effect.
5. When completing L2 to L1 backfill:
    - The sPOL selling happens with an outdated L1 exchange rate, unfavorable to the user.
    - In the controller `startBackfillSell` burns more sPOL than what is needed to get the required POL.
    - The larger the discrepancy between the L1 and L2 exchange rates, the more excess sPOL is burned, and the greater the rate inflation on L1.

The first two points can only be triggered by the admin, and thus require extreme caution. A safe way to proceed would be to perform the following steps:

- Pause both L1 and L2 contracts.
- Trigger any migration or backfill that can be performed, ensuring that there is no surplus POL remaining on L2, nor pending withdrawals to be processed with a backfill.
- Perform the risky operation (reload validators or clean up MATIC POL).
- Bridge the new exchange rate to L2.
- Unpause both L1 and L2 contracts.

The third point, stealth donations, only becomes material when sPOL supply is very low, which is expected not to happen if the Polygon team mints and permanently locks a substantial initial sPOL supply on L1 (see Trust Model and First Depositor Inflation Attack).

The last two points are more problematic, since, while a migration or a backfill can only be triggered by the admin, the L2 state that triggers one of these operations can be influenced by end users. How they can be used to create an unexpected appreciation of the L1 exchange rate is described below.

Effect:

There are two main impacts of an unexpected appreciation of the L1 exchange rate:

1. It can brick migrations from L2 to L1, as the check in `sPOLController.completeMigration` (which enforces that the L1-computed expected sPOL amount is not less than the sPOL amount minted on L2) would revert:

```
uint256 expectedSPOL = convertPOLtoSPOL(_amountPOL);
require(
    expectedSPOL >= _amountSPOL,
    BadExchangeRate(_amountPOL, _amountSPOL, totaldPOLBalance - feedPOLBalance, totalsPOLBalance())
);
```

2. It can lead to arbitrage when users buy sPOL on L2 at an exchange rate that is lower than the current L1 exchange rate, even after accounting for the safety fee.

Attack 1: Bricking a migration

1. The `sPOLController` minted `1000 * 10^18` sPOL for `1000 * 10^18` dPOL, with an exchange rate of 1:1.
2. The L1 exchange rate has been bridged to L2, and is still 1:1.
3. The `sPOLChild` is initially empty, for simplicity.
4. With the safety fee `f = 0.3%`, a user buys `1994 * 10^18` sPOL for `2000 * 10^18` POL, and immediately sells them back. This leaves the child with a surplus of `6 * 10^18` POL and `locallyMintedSPOL - locallyToBeBurnedSPOL = 0`.
5. The admin calls `balanceWithL1`, which triggers a migration of `6 * 10^18` POL for `0` sPOL.
    - The new `sPOLController` state becomes `totaldPOLBalance = 1006 * 10^18`, `totalsPOLBalance() = 1000 * 10^18`.
6. In the meantime, users buy more sPOL on L2, still at the old rate of 1:1, say X sPOL for Y POL, where `X == Y * 0.997`.
7. The admin now bridges the L1 exchange rate to L2, which triggers a migration of Y POL for X sPOL.
    - In `sPOLController.completeMigration`, the expected sPOL amount is calculated as `expectedSPOL = convertPOLtoSPOL(Y)`, which, given the new exchange rate of `1.006:1`, leads to `expectedSPOL = Y / 1.006`, which is less than `X = Y * 0.997` (excluding rounding for low Y), leading to a revert.
8. At that point, the system is bricked with a pending migration that cannot be completed. One way to recover from this state is to donate to the `sPOLChild` enough sPOL to cover the migration, by bridging the exact amount of sPOL needed from L1 to L2, see Migration completion can be front-run for more details.

Note that, although this can be performed by an attacker, this could also happen via honest user activity if the `sPOLChild` contract is being heavily used for buys and sells, leading to unexpected appreciation of the L1 exchange rate.

The exact condition for this to happen is as follows:

Let `R_old` be the L1 rate used on L2 when minting (`dPOL/sPOL`), and `f` the safety fee. A migration reverts when, given `X` the amount of POL being migrated, the following does not hold `X / R_now >= X(1-f) / R_old`.

Attack 2: Arbitraging the price difference

This attack is rather similar to the previous one, but instead of bricking a migration, the attacker aims to profit from a stale L2 exchange rate after L1 has appreciated beyond the safety fee.

- Steps 1 to 6 are similar to the previous attack, leading to an appreciated L1 exchange rate on L1 while L2 still uses the older rate.
- Let `R_old` be the rate used on L2 and `f` the safety fee. Buying sPOL on L2 costs `R_old/(1 - f)` POL per sPOL, while selling on L1 returns `R_now` POL per sPOL. The trade is profitable when `R_now > R_old/(1 - f)` (the same condition that makes `completeMigration` revert).
- The attacker buys X sPOL on L2 for Y dPOL, bridges X sPOL to L1, and sells them back for Z POL where `Z > Y`.

Note that this attack is limited by the liquidity locked in the L1 side of the sPOL bridge.

Additional Note:

While migrations/backfills can only be triggered by the admin, control over the timing of a migration/backfill only provides limited protection and does not mitigate this issue. Once a dangerous `sPOLChild` state is reached (driven by user activity), the way to prevent the migration/backfill from committing that state to the L1 side is unclear, which justifies the severity level assigned to this issue.

Code corrected:

Several code and design changes have been made to address this issue. The core architectural change replaces the privileged `completeMigration` and `startBackfillSell` functions in the controller with a design where the messenger operates as a normal user, calling `buySPOL` and `sellSPOL` on the controller at the current L1 rate.

Migrations (point 4): In Version 1, `completeMigration` staked the full bridged POL but only minted the sPOL amount computed on L2 with the safety fee discount, inflating the L1 rate by the difference. In Version 2, `_handleMigration` computes `requiredPOL` as `convertSPOLtoPOL(_mintedSPOL) + 1` at the current L1 rate and calls `buySPOL(requiredPOL)`, effectively performing a buy with the controller. Only the required POL enters `totaldPOLBalance`; the surplus is transferred to the controller as unstaked POL. The `+1` compensates for the floor division in `convertSPOLtoPOL`, ensuring the subsequent `convertPOLtoSPOL` inside `buySPOL` yields at least `_mintedSPOL`. Since the migration now executes at the current L1 rate rather than at the discounted L2 rate, no excess POL is staked relative to the sPOL minted, eliminating the rate inflation previously caused by migrations.

Backfills (point 5): In Version 1, `startBackfillSell` burned the sPOL amount computed at L2's stale rate but only unstaked a smaller POL amount, burning more sPOL than necessary. In Version 2, the messenger calls `sellSPOL` at the current L1 rate. Any surplus POL after covering the requested amount is sent to the controller as unstaked balance. Since the backfill now sells at the current L1 rate rather than at L2's outdated rate, no excess sPOL is burned relative to the POL withdrawn, eliminating the rate inflation previously caused by backfills.

These changes significantly mitigate the attacks described above, as migrations and backfills no longer inflate the L1 rate as a side effect.

Points 1, 2, and 3 remain unchanged in design. Admin operations (`reloadValidatorInfo`, `cleanUpMaticPOL`) and stealth donations still carry the same risks described in the original issue and must be handled with the same operational caution.

Operational note: The surplus POL from migrations and backfills now accumulates as unstaked balance in the controller. This balance only enters `totaldPOLBalance` when the admin calls `cleanUpMaticPOL`, which stakes POL without minting sPOL and is therefore a rate-inflating operation. After many migration and backfill cycles, substantial amounts of POL may accumulate if not handled regularly. The admin must treat `cleanUpMaticPOL` with caution.

Recovery note: If the L1 exchange rate appreciates beyond the safety fee before a migration message is processed on L1, the migration can still revert due to insufficient POL. Recovery is possible by donating enough POL to the messenger contract, which can then be used to complete the migration.

Similarly, if the L1 exchange rate depreciates and a backfill reverts due to insufficient sPOL, recovery is possible by donating enough sPOL to the messenger contract.

Rounding note: The `+1` in the required POL calculation for migrations is a simple way to ensure that enough sPOL is minted despite rounding errors. However, it can result in slightly more sPOL being minted than what is then bridged to L2 because of conversion rounding. In that case, the excess sPOL remains in the messenger and can be used in the next backfill.

6.3 Missing `polBalance` Decrease During Migration

Design | High | Version 1 | Code Corrected

CS-POLYGON-SPOL-003

The migration flow of `sPOLChild` is initiated in `_balanceWithL1` using `_requestMigration`. Surplus POL is sent to the L2 `polBridger` via `_exitPOLforMessenger()`:

```
function _requestMigration(uint256 _polToMigrate, uint256 _spolToMint) internal {
    onGoingMigration = true;
    backMigratingSPOL = _spolToMint;

    _exitPOLforMessenger(_polToMigrate);
    ...
}
```

However, the local accounting variable `polBalance` is never updated to reflect the transfer. This leaves the child contract in an inconsistent state where it assumes it holds a surplus POL balance that has already been sent away. In practice, this will:

- Let follow-up migrations incorrectly use users' `reservedWithdrawPOLBalance`
- Prevent backfills from occurring, as the surplus POL is consistently overestimated
- Eventually deplete the contract's POL balance and cause `_balanceWithL1` to consistently revert once requested migrations exceed the contract's actual POL balance.

Code corrected:

The `_requestMigration` function was updated to decrease the `polBalance` by the migrated amount, ensuring that the contract's internal accounting remains consistent with the actual POL balance after migrations.

6.4 `sPOLChild` Missing Fallback to Receive POL

Design | High | Version 1 | Code Corrected

CS-POLYGON-SPOL-004

The `sPOLChild` contract does not implement a payable `receive()` or `fallback()`. As a result, any native POL transfer sent with empty calldata will revert.

This is problematic for the backfill flow: after a successful backfill, when `polToken.deposit()` is invoked by the childChain contract, the following call to the `sPOLChild` reverts:

```
(bool success, bytes memory ret) = receiver.call.value(amount).gas(txGasLimit)("");
if (!success) {
    assembly {
        revert(add(ret, 0x20), mload(ret)) // bubble up revert
    }
}
```

This means that once a backfill is initiated, the POL tokens cannot be deposited into the `sPOLChild` contract. The contract then becomes insolvent given its internal accounting of POL balances versus actual POL held, and cannot process all withdrawals correctly.

Code corrected:

The `sPOLChild` contract now has a `receive()` function to handle native token transfers.

6.5 First Depositor Inflation Attack

Security | Low | Version 1 | Specification Changed

CS-POLYGON-SPOL-008

The `sPOLController` is vulnerable to a first-depositor inflation attack. An attacker can manipulate the exchange rate through stealth donations (deposits that mint zero sPOL due to rounding), artificially inflating the price of sPOL. This allows the attacker to front-run legitimate depositors and cause them to receive zero sPOL for their deposit.

Both `convertPOLtoSPOL` and `convertSPOLtoPOL` perform divisions that round down:

```
function convertPOLtoSPOL(uint256 _amountPOL) public view returns (uint256) {
    // ...
    return _amountPOL * currentTotalsPOLBalance / (totaldPOLBalance - feedPOLBalance);
}
```

When the system has low liquidity, an attacker can exploit this rounding by:

1. Seeding the pool with a minimal deposit (1 wei POL → 1 wei sPOL)
2. Triggering a restake to create a state where `totaldPOLBalance > totalsPOLBalance`
3. Repeatedly calling `buySPOL` with amounts that round down to 0 sPOL, inflating the backing per sPOL
4. Once the exchange rate is sufficiently inflated, a victim's deposit (e.g., 10 POL) mints 0 sPOL
5. The attacker redeems their single sPOL to claim the victim's deposit

Similarly, other ways of inflating the exchange rate could be used to front-run legitimate first depositor users of the controller. Those vectors are listed in the L1 Exchange-Rate Appreciation Can Brick L2 Migrations or Enable Arbitrage issue.

Particularly, stealth donations in the `sPOLChild` followed by a migration to L1 could achieve a similar behavior as the attacker could perform stealth donations repeatedly, using the safety fee as a stealth-donation amplifier, before a migration that realizes the inflated exchange rate on L1.

The manipulated exchange rate can also be propagated to Polygon via bridging.

This issue is noted as low severity given the strong Trust Model assumption that the Polygon team will mint and lock a substantial initial sPOL supply on L1, preventing low-liquidity states that facilitate this attack. This implies:

- The initial sPOL supply should be large enough to prevent meaningful exchange-rate manipulation via stealth donations.
- The initial sPOL supply should be permanently locked to avoid re-initialization attacks.
- The initial sPOL supply should *NOT* be bridged to L2, as anyone could mint the corresponding amount on L2 and bridge it back to L1 to drain the initial liquidity out of the Controller.

However, even with an initial deposit, inflation by stealth donations remains possible, albeit at a much higher cost to the attacker that is dependent on the size of the initial deposit and likely without profit. This is especially true since there are No slippage protections to protect users from such manipulations.

Specification changed:

The Polygon team acknowledged this risk and formalized the mitigation in the trust model: a sufficiently large initial sPOL supply is minted at a 1:1 rate, permanently locked, and never bridged to L2. This prevents low-liquidity states that would enable this attack in practice.

6.6 Incomplete Validator Status Checks on `migrateValidator`

Design | Low | Version 1 | Specification Changed

CS-POLYGON-SPOL-009

Within `sPOLController.migrateValidator`, the validator status checks are contingent on the `_restake` parameter. If `_restake` is set to `false`, these validations are bypassed, potentially allowing invalid state transitions.

This could lead to cases where:

- The sum of `validators[v].totalStaked` for all active or frozen validators does not equal `totaldPOLBalance`.
- An active validator is migrated to a validator that is not supported by the controller.
- The exchange rate decrease (migrate to a validator not supported by the controller before reloading validators).

Specification changed:

Polygon accepts this behavior and specified that it is the caller's responsibility to ensure correctness when using `_restake=false` and answered:

```
Intentional — _restake=false bypasses status checks to allow recovery
migrations from deactivated validators. NatSpec updated to warn callers
that they must ensure correctness when using _restake=false.
```

6.7 Incorrect Value Logged in `BackfillCompleted` Event

Correctness | Low | Version 1 | Code Corrected

In the sPOLMessenger contract, the function completeBackfill logs the following event:

```
completedBackfill[currentActiveBackfillCycle] = true;
currentActiveBackfillCycle = 0;
emit BackfillCompleted(currentActiveBackfillCycle, totalWithdraw);
```

The event BackfillCompleted is emitted with the parameter currentActiveBackfillCycle, which has just been set to 0. This means that the logged value for currentActiveBackfillCycle will always be 0, regardless of which backfill cycle was actually completed.

Code corrected:

The code now caches currentActiveBackfillCycle in a local variable, processingBackfillCycle, at the start of completeBackfill. The event is emitted using this cached value, so the log reflects the actual completed cycle. currentActiveBackfillCycle is then reset to 0.

6.8 Missing depositShare Validation on Validator Removal

Design | Low | Version 1 | Code Corrected

sPOLController.removeValidator lacks validation to ensure the depositShare of the remaining validators still sums to 100% after removal.

When this invariant is violated, subsequent calls to sPOLController._validatorWithHighestTotalStakeDistance() may incorrectly return type(uint256).max. This can propagate inaccurate distance values to integrators that depend on sPOLController.getMostUnderfundedValidator(), potentially causing malfunctions or unintended validator selection.

Note that the invariant is also violated when the first validator is added, since depositShare is not updated to 100% and remains at 0%.

Code corrected:

removeValidator now requires depositShare == 0 before removal. Only updateValidatorTargetShare can modify deposit shares, and it enforces the 100% sum invariant.

Accordingly, the sum of depositShare across all validators is maintained at 100%, except in the initial state after adding the first validator and before the first call to updateValidatorTargetShare.

6.9 No Minimum Safety Fee Validation

Design | Low | Version 1 | Code Corrected

In sPOLChild, changeSafetyFee does not validate that the new safety fee is above a minimum threshold. A zero or extremely low safety fee could compromise security and enable attacks described in L1 Exchange-Rate Appreciation Can Brick L2 Migrations or Enable Arbitrage at a lower cost.

Code corrected:

The code now validates that _newFee is greater than 0, rejecting zero-fee configurations.

6.10 `reloadAllValidatorInfo` Reloads Deactivated Validators

Design | Low | Version 1 | Code Corrected

CS-POLYGON-SPOL-015

In the controller, the function `reloadAllValidatorInfo` specifies:

```
// very expensive, includes frozen validators
```

However, the function also reloads deactivated validators. Callers should be aware of this behavior, as it can make the system appear insolvent: the newly reloaded validator shares are added to `totaldPOLBalance` but cannot be sold. The admin can recover by migrating the shares (with `restake = false`) to an active validator, which may be subject to the issue raised in Incomplete validator status checks on migrateValidator.

Code corrected:

The function `reloadAllValidatorInfo` was removed from the codebase. Instead, a single-validator function, `reloadValidatorInfo(uint16)`, was added for targeted reloads, with NatSpec warning against reloading inactive validators (the freezing feature was removed from the codebase).

6.11 `removeValidator` Can Be Temporarily DoSed

Design | Low | Version 1 | Code Corrected

CS-POLYGON-SPOL-016

`removeValidator` relies on the following requirement:

```
uint256 shares = removedValidator.validatorContract.balanceOf(address(this));
require(shares == 0, ValidatorSharesPending(_removedValidator, shares));
```

By donating validator shares to the controller, an attacker can prevent removal of that validator until the donated shares are cleared. Clearing the shares requires a privileged party to:

1. Reload that validator using `reloadAllActiveValidatorInfo`
2. Call `migrateValidator` to move the shares to a different validator

Because reloading cannot be done for a single validator, a privileged party must reconcile balances across all active validators. This can sharply inflate the sPOL price, which is undesirable given the assumption that sPOL should not spike to avoid an exchange rate so high that migrations and backfills cannot complete, as depicted in L1 Exchange-Rate Appreciation Can Brick L2 Migrations or Enable Arbitrage.

Code corrected:

The `balanceOf` check in `removeValidator` was removed, mitigating this DoS vector. Removal now requires both `totalStaked == 0` and `depositShare == 0`, ensuring protocol shares are fully unstaked and the validator's target allocation is zeroed before removal.

Users can still attempt to buy SPOL in this validator right before removal, but the admin can migrate those shares to another validator before removing it, without the side effects described in the original issue, mitigating the DoS risk.

6.12 `sPOLBurned` Event Logs Incorrect Amount In Multi-Validator Sell

Correctness **Low** **Version 1** **Code Corrected**

CS-POLYGON-SPOL-017

In `_sellSPOLMulti`, the `sPOLBurned` event logs an incorrect `sPOLAmount` for all loop iterations after the first. Each call to `_sellSharesFromValidator` modifies `totaldPOLBalance` through restaking, effectively changing the exchange rate. This is not accounted for when computing `sPOLAmount` inside the loop.

As a result, the `sPOLBurned` event logs an `sPOLAmount` that does not correspond to the actual amount of sPOL burned for all but the first validator in the loop. Off-chain systems or integrators relying on `sPOLBurned` events to track sPOL amounts per withdrawal will receive incorrect data.

Code corrected:

`sPOLAmounts` are now computed in a separate loop before the sell loop, using the same exchange rate for all validators. Note, however, that due to rounding, the sum of individual `sPOLAmounts` may not exactly match the total sPOL burned. Still, each value remains consistent with the sPOL-to-POL exchange rate at sell time.

6.13 Division-by-zero in Redeem Distance Math

Informational **Version 1** **Code Corrected**

CS-POLYGON-SPOL-046

In `sPOLController`, the internal `_maxRedeem` function computes the maximum redeemable amount for a validator based on `depositShare` and `maxDivergence`.

When `maxDivergence == 0` and a validator has `depositShare == 100`, the function computes:

```
uint8 mySmallShare = validatorShare - currentMaxDivergence;
uint8 restShare = 100 - mySmallShare;
uint256 myactualMinShare = (theOtherShare * mySmallShare) / restShare;
```

In this edge case, `mySmallShare` becomes `100`, so `restShare` becomes `0`, which causes a division-by-zero revert.

Code corrected:

The function now includes a check to prevent division by zero. If `restShare` is zero, it returns `validatorStaked`.

6.14 Exchange-rate Decrease

Informational **Version 1** **Code Corrected**

CS-POLYGON-SPOL-019

The exchange rate should be non-decreasing for the system to function correctly. However, it can decrease in the following two cases:

1. Nothing prevents the controller from being emptied if all users sell their sPOL tokens. However, the Polygon team plans to mint an initial sPOL supply and burn/lock it to prevent this from happening, as detailed in the Trust Model section of this report.
2. The admin, even though trusted not to, could exploit Incomplete validator status checks on migrateValidator and perform the following sequence of actions, assuming `V0` is `ACTIVE` and `V1` is `INACTIVE`:
    - `migrateValidator(V0, V1, X, false)`
    - `reloadAllValidatorInfo()`

A decreasing exchange rate would permanently break the system, as the following functions would revert:

- `_handleExchangeRateUpdate`
- `startBackfillSell`

Code corrected:

Although it should never happen, the code was updated in version 2 so exchange-rate decreases no longer cause permanent system failure.

- `_handleExchangeRateUpdate` checks whether the incoming rate is lower than the current rate. If it is, the function emits `ExchangeRateDeclined` and returns without reverting, preserving the previous (higher) rate and keeping the system operational.
- Backfills may still fail due to an exchange-rate decrease, but in the new design, donating sPOL to the messenger can help recover from that state.

6.15 Function Visibility Too Broad

Informational **Version 1** **Code Corrected**

CS-POLYGON-SPOL-020

The following functions are declared as `public` but could be `external` (variants that also take the validator as parameter):

- `sPOLController.buySPOL`
- `sPOLController.buySPOLPermit`

Code corrected:

The visibility of these functions was changed to `external`.

6.16 Gas Savings

Informational | Version 1 | Code Corrected

CS-POLYGON-SPOL-021

The following non-exhaustive list of gas inefficiencies have been identified in the system:

sPOLController:

1. In `removeValidator`, `removeValidator.validatorContract` is read twice from storage, it could be cached instead.
2. In `_removeFromActiveValidators`, the length of `activeValidators` is read twice at each iteration of the loop, it could be cached instead.
3. In `updateValidatorTargetShare`, `restakeAllActiveValidators`, `reloadAllActiveValidatorInfo`, `_selectValidators` and `_validatorWithHighestTotalStakeDistance` the length of `activeValidators` is read at each iteration of the loop, it could be cached instead.
4. In `reloadAllValidatorInfo` the length of `validatorList` is read at each iteration of the loop, it could be cached instead.
5. In `reloadAllActiveValidatorInfo` and `reloadAllValidatorInfo`, `validator.totalStaked` is written to storage before being read again, a cached variable could be used instead.
6. In `_withdrawPOL`, `validator.validatorContract` and `currentDetails.validatorNonce` are read from storage multiple times, they could be cached instead.
7. In `_maxDeposit`, `validator.totalStaked` is read from storage multiple times, it could be cached instead.
8. In `_maxRedeem`, `_validator.depositShare`, `_validator.totalStaked` and `maxDivergence` are read from storage multiple times, they could be cached instead.
9. In `_selectValidators`, `activeValidators.length` and `validators[activeValidators[i]].totalStaked` are read multiple times, they could be cached instead.
10. In `_takeFee`, `feeReceiver` and `feedPOLBalance` are read from storage multiple times, they could be cached instead.
11. In `changeFeeReceiver`, `feeReceiver` is used for the event emission although a cached variable could be used instead.
12. In `changeRewardFee`, `rewardFee` is used for the event emission although a cached variable could be used instead.
13. In `changeMaxDivergence`, `maxDivergence` is used for the event emission although a cached variable could be used instead.

sPOLMessenger:

1. In `completeBackfill`, `currentActiveBackfillCycle` and `backfillAmounts[currentActiveBackfillCycle]` are read from storage multiple times, they could be cached instead.

sPOLChild:

1. In `sellSPOL`, `globalWithdrawNonce` is read three times from storage, it could be cached instead.

2. In `withdrawPOL`, `currentOutstanding.outstandingPOL` is read twice from storage, it could be cached instead.
3. In `_burnSPOLForMessenger`, `l1Messenger` is read several times from storage, it could be cached instead.
4. In `_balanceWithL1`, `missingWithdrawPOLBalance` is read several times from storage, it could be cached instead.
5. In `_startBackfill`, `backFillCycle` is used for the event emission although a cached variable could be used instead.
6. In `_requestBackfill`, `backFillCycle` is read several times from storage, it could be cached instead.

Code corrected:

- **sPOLController:** Items 1-9 and `feedPOLBalance` in item 10 have been addressed, other points intentionally left as-is.
- **sPOLMessenger:** Item 1 has been cached.
- **sPOLChild:** Items 1-3 have been cached, other points intentionally left as-is.

6.17 Inconsistent Reentrancy Protection

Informational **Version 1** **Code Corrected**

CS-POLYGON-SPOL-022

In `sPOLChild`, reentrancy protection is not consistent across functions. Some user-facing functions are protected with the `nonReentrant` modifier, while others are not. For example, `buySPOL`, `deposit` and `withdraw` are not protected, while `sellSPOL` and `withdrawPOL` are.

Similarly, in `sPOLController`, only `completeMigration` and `startBackfillSell` are protected with the `nonReentrant` modifier.

Code corrected:

The `nonReentrant` modifier was added to all user-facing functions in both `sPOLChild` and `sPOLController`, except for `sPOLChild.withdraw()`. This ensures consistent reentrancy protection across critical entry points.

6.18 Incorrect Interface

Informational **Version 1** **Code Corrected**

CS-POLYGON-SPOL-023

The interface `IsPOLController` defines multiple functions that do not match the implementation in `sPOLController`. This mismatch is not caught by the compiler because `sPOLController` does not inherit from the interface.

Code corrected:

The `IsPOLController` interface has been removed from the codebase.

6.19 Missing Comprehensive NatSpec Documentation

Informational | Version 1 | Code Corrected

CS-POLYGON-SPOL-025

The contracts lack comprehensive NatSpec documentation. In particular, many functions and their input/output parameters are undocumented, and several critical behaviors are left without clear descriptions. This reduces clarity for integrators and reviewers, and increases the risk of incorrect usage or assumptions about edge cases.

Code corrected:

NatSpec documentation was added across all core contracts (`sPOLController`, `sPOLChild`, `sPOL`, `polBridger`, `sPOLMessenger`), including contract-level descriptions, function intent, parameter and return-value semantics, and developer notes on side effects and access control.

6.20 Missing or Superfluous Events

Informational | Version 1 | Code Corrected

CS-POLYGON-SPOL-027

The following functions modify critical state but do not emit corresponding events. Adding events can improve transparency and facilitate off-chain monitoring.

- All functions in `polBridger`
- In the `else` branch of `sPOLChild.deposit()`, the event `sPOLMinted` is not emitted after minting sPOL.
- In `sPOLChild.withdraw()`, the event `sPOLBurned` is not emitted after burning sPOL.
- In `sPOLController.completeMigration()`, `sPOLMinted` is not emitted upon minting sPOL.
- In `sPOLController`, while `_mintSPOL` emits `sPOLMinted`, `_takeSPOL` does not emit any event.

The following events are superfluous:

- In `restakeAllActiveValidators`, `_emitExchangeRateUpdate` is called twice at the end of the loop without any state change in between, leading to duplicate events with the same data.

Code corrected:

The code has been updated as follows:

- `polBridger` functions remain without dedicated events, because they operate on behalf of main contracts that already emit their own events alongside POL token `Transfer` events.
- `sPOLChild.deposit()` and `sPOLChild.withdraw()` do not emit `sPOLMinted` or `sPOLBurned`, since they move existing sPOL rather than creating or destroying supply.
- `completeMigration()` was removed. Migrations are now processed via `_buySPOLMulti()`, which calls `_mintSPOL()` and emits `sPOLMinted`.
- `_takeSPOL()` remains without a dedicated event because the caller already emits the relevant event.

- The superfluous `_emitExchangeRateUpdate()` call was removed.

6.21 Non-indexed Event Parameters

Informational **Version 1** **Code Corrected**

CS-POLYGON-SPOL-029

Several events in the sPOL contracts omit indexed parameters. Indexing relevant fields (e.g., user addresses or validator IDs) makes it cheaper and faster for integrators and dApps to filter logs. The list below is non-exhaustive:

sPOLChild:

- `sPOLMinted`: address `user`
- `sPOLBurned`: address `user`
- `POLWithdrawn`: address `user`
- `BackfillCompleted`: uint256 `backfillCycle`
- `BackfillLocalCompleted`: uint256 `backfillCycle`
- `BackfillRequested`: uint256 `backfillCycle`
- `BackfillStarted`: uint256 `backfillCycle`

sPOLController:

- `ValidatorAdded`: uint16 `validatorId`
- `ValidatorRemoved`: uint16 `validatorId`
- `ValidatorFrozen`: uint16 `validatorId`
- `ValidatorUnfrozen`: uint16 `validatorId`
- `ValidatorTargetShareChanged`: uint16 `validatorId`
- `ValidatorMigrated`: uint16 `oldValidator`, `newValidator`
- `sPOLMinted`: address `user`
- `sPOLBurned`: address `user`
- `POLWithdrawn`: address `user`
- `sPOLMigrated`: address `user`
- `sPOLBackfilled`: address `user`
- `FeeCollected`: address `feeReceiver`
- `FeeReceiverChanged`: address `oldReceiver`, `newReceiver`
- `POLTokensCleaned`: uint16 `validatorId`

Code corrected:

- The `sPOLMinted`, `sPOLBurned`, and `POLWithdrawn` events in `sPOLChild` and `sPOLController` have been updated to index the user address.
- The backfill events in `sPOLChild` and fee-related events in `sPOLController` remain unindexed.
- The `sPOLMigrated` and `sPOLBackfilled` events have been removed.

- For events related to validator IDs, Polygon states:

```
Validator IDs and system operation events not indexed -
system operations are tracked holistically, not per-validator.
```

6.22 Outdated or Pending Comments

Informational **Version 1** **Code Corrected**

CS-POLYGON-SPOL-030

The following comments appear outdated, no longer relevant, or left pending review. Updating or removing them can improve code clarity.

```
// These three together should always be equal to the sum of outstanding POL in userOutstandingPOL

// maybe don't revert here to avoid failedStateSync issues

// this then stays in failedstatesync, maybe don't revert, but ignore?

// disabled for portal because of cyclical exit issue, should be activated for lxly
//_sendMessageToChild(abi.encode(MsgType.L1_MIGRATION_RESPONSE, encodeL1MigrationResponseMessage(_mintedSPOL)));

// formerly lastSuccessfulBuyValidator and lastSuccessfulSellValidator

// Since it is called via a system call, any event will not be emitted during its execution.
```

Code corrected:

The lxly comment in the messenger is intentionally kept as a reference for a future bridge upgrade. All other comments were removed or updated.

6.23 Solidity Version Not Fixed

Informational **Version 1** **Code Corrected**

CS-POLYGON-SPOL-032

The Solidity compiler version is not explicitly fixed, neither in the contracts nor in the `foundry.toml` file. This can make reproducing builds difficult in the future when newer compiler versions are released.

Code corrected:

The Solidity version was explicitly set to `0.8.30` in all contracts.

6.24 Typo in Variable Names

Informational **Version 1** **Code Corrected**

CS-POLYGON-SPOL-033

- In `sPOLController`, the variable name `myactualMinSahre` is misspelled.
- In `sPOLController`, the event parameter `totalbPOLBalance` of the `ExchangeRateSnapshot` event is misspelled.

Code corrected:

The first spelling issue was corrected. The second case is intentional, as the b prefix stands for "backing POL."

6.25 Unused Errors

Informational | Version 1 | Code Corrected

CS-POLYGON-SPOL-034

Several errors are defined but never emitted in the contract logic. Removing these unused definitions would help to improve code clarity.

sPOLController:

- `NonceNotFound`
- `AmountTooLarge`

sPOLChild:

- `NothingToBackfill`
- `NothingToBalance`
- `NothingToMigrate`

Code corrected:

Unused error definitions were removed from both `sPOLController` and `sPOLChild`.

6.26 Unused Imports

Informational | Version 1 | Code Corrected

CS-POLYGON-SPOL-035

The following import is declared in the code but is not used anywhere. Removing unused imports improves readability.

- In `sPOLChild`, `IERC20`

Code corrected:

The unused import has been removed.

6.27 Unused Variable `spacer` in `sPOLController`

Informational | Version 1 | Code Corrected

CS-POLYGON-SPOL-036

In `sPOLController`, the variable `spacer` appears to be legacy code that was replaced but not removed. Removing it would help to improve code clarity and reduce deployment costs.

Code corrected:

The unused `spacer` variable was removed from `sPOLController`.

6.28 Use of State Variable Instead of Function Parameter

Informational | Version 1 | Code Corrected

CS-POLYGON-SPOL-037

In `sPOLChild._requestBackfill()` the following is done:

```
function _requestBackfill(uint256 _polToBackfill, uint256 _spolToSell) internal {
    requestedWithdrawPOLBalance += missingWithdrawPOLBalance;
    ...
}
```

While in the current version of the code, `missingWithdrawPOLBalance` is always equal to `_polToBackfill`, this may not always hold if the code changes. It would be clearer to use the function parameter directly.

Code corrected:

`_requestBackfill` now uses the `_polToBackfill` function parameter instead of the state variable.

6.29 `depositData` Encoding

Informational | Version 1 | Code Corrected

CS-POLYGON-SPOL-039

In `sPOLMessenger._handleMigration`, `rootChainManager.depositFor` is called with `abi.encodePacked(_mintedSPOL)`. While `abi.encodePacked(uint256)` is currently equivalent to `abi.encode(uint256)`, the bridge expects abi encoded data, and the packed form might diverge if the payload ever changes (for example, adding parameters). Using `abi.encode` makes the intent explicit and avoids future ambiguity.

Code corrected:

The code has been changed to use `abi.encode`.

6.30 `rescue` Cannot Transfer POL Tokens

Informational | Version 1 | Code Corrected

CS-POLYGON-SPOL-040

In the `polBridger`, the function `rescue` is designed to allow the contract owner to transfer arbitrary tokens from the contract to a specified recipient. However, it does not support transferring POL tokens on L2, given that it is the native token and not a standard ERC20 token.

Code corrected:

To enable native POL recovery, a new `rescueNative` function was added to `polBridger`.

6.31 `rescue` Incompatible With Non-Compliant ERC20 Tokens

Informational | Version 1 | Code Corrected

CS-POLYGON-SPOL-041

In the `polBridger`, the function `rescue` allows the contract owner to transfer arbitrary tokens from the contract to a specified recipient. The transfer does not use `safeTransfer`, and will fail for non-compliant ERC20 tokens such as USDT that do not return a boolean value on transfer.

Moreover, although `POL` and `sPOL` are compliant ERC20 tokens, other ERC20 operations in the codebase could also use safe methods for the sake of consistency and security.

Code corrected:

The `polBridger` contract now imports `SafeERC20` and uses `safeTransfer` in `rescue`. Other ERC20 operations were not updated to use `safeTransfer`.

7 Informational

We utilize this section to point out informational findings that are less severe than issues. These informational issues allow us to point out more theoretical findings. Their explanation hopefully improves the overall understanding of the project's security. Furthermore, we point out findings which are unrelated to security.

7.1 `getMostUnderfundedValidator` Can Return Misleading Values

Informational **Version 2** **Acknowledged**

CS-POLYGON-SPOL-045

In `sPOLController`, the internal `_validatorWithHighestTotalStakeDistance` function is designed to identify which validator has the greatest `_validatorMaxTotalStakeDistance`. It achieves this by iterating through active validators, calculating the distance for each, and retaining the validator with the highest distance encountered.

```
for (uint256 i = 0; i < activeValidatorsLength; i++) {
    ValidatorInfo storage validator = validators[activeValidators[i]];

    // Skip locked validators when looking for deposit capacity
    if (_positive && validator.validatorContract.locked()) {
        continue;
    }

    uint256 distance = _validatorMaxTotalStakeDistance(validator, _positive);

    if (maxDistance < distance) {
        maxDistance = distance;
        selectedValidator = validator.index;
    }
}
```

In version 2, a critical check was introduced to bypass locked validators, thereby preventing attempts to stake to validators that are not accepting new deposits. However, an edge case arises when all underfunded validators are locked or whenever no unlocked validator has `distance > 0`: the loop completes without identifying an eligible candidate, so `maxDistance` is never updated and retains its default value of 0. It then explicitly sets the return value to `type(uint256).max`.

```
if (_positive && maxDistance == 0) {
    maxDistance = type(uint256).max;
}
```

The function returns `type(uint256).max`, and this is misleading because the returned validator `validators[activeValidators[0]]` is either locked or already overfunded.

Acknowledged:

Polygon acknowledged this issue and answered:

```
This case (all underfunded validators locked) should not occur in practice.
Even if it does, only affects off-chain consumers of the view function.
On-chain buy/sell paths via `_selectValidators` handle locked validators
correctly. No on-chain harm.
```

7.2 Absence of Nonce-Based Withdrawals

Informational **Version 1** **Acknowledged**

CS-POLYGON-SPOL-042

The issue Fund lockup due to inconsistent validation in sellSPOL and withdrawPOL is severe because users cannot skip nonces in the pending withdrawals queue.

Allowing users to withdraw specific nonces (or a range of nonces) would make the system more flexible and resilient, so a single problematic entry does not block later withdrawals.

Acknowledged:

Polygon answered:

```
Previously implemented but removed as the UI team had no use for it.
The CS-001 fix prevents zero-share nonces from being created.
A locked withdrawal would be a critical bug in both sPOL and the
StakeManager system — we would fix and recover user funds via
upgrade if this ever occurred.
```

7.3 Dead Code

Informational **Version 1** **Acknowledged**

CS-POLYGON-SPOL-018

`MsgType.L1_MIGRATION_RESPONSE` and related logic in `sPOLChild` and `MsgCoder` appear to be remnants of a previous migration mechanism that is no longer active. Removing dead code can improve maintainability and reduce confusion.

Acknowledged:

Polygon answered:

```
Kept intentionally. The L1_MIGRATION_RESPONSE enum value and
related encode/decode functions serve as reference for a future
lxly bridge upgrade.
```

7.4 Migration Completion Can Be Front-Run

Informational **Version 1** **Acknowledged**

CS-POLYGON-SPOL-024

The migration completion check in `sPOLChild.deposit` relies on the public tuple `(user == address(this), onGoingMigration, amount == backMigratingSPOL)`. A third party can therefore front-run the legitimate migration deposit by bridging exactly `backMigratingSPOL` to the `sPOLChild` address, which marks the migration as complete.

This is not an exploitable path nor an issue. Any party doing so is effectively donating their sPOL to the contract and receives no value in return.

Acknowledged:

Polygon acknowledged this behavior and does not consider it an issue.

7.5 Missing Sanity Checks

Informational **Version 1** **Code Partially Corrected**

CS-POLYGON-SPOL-026

Several functions across the contracts are missing sanity checks on input parameters and state conditions. Implementing these checks can prevent unintended behavior and improve robustness.

1. `_handleBackfillResponse` does not check `returnedBackFillCycle` against the current cycle.
2. In `sPOLController.initialize`, no check is performed on `_maxDivergence`.
3. In `sPOLChild.sellSPOL`, no check is performed to ensure the freshness of the exchange rate.
4. In `sPOLController._buySPOLSingle`, the return value of `_buySharesFromValidator` is not ensured to be equal to `_amount` as done in `_buySPOLMulti`.
5. In `sPOLController.updateValidatorTargetShare`, no deduplication is done on the provided `_validatorID` array.
6. In `sPOLChild`, `setMaxExchangeRateUpdateDelay` has no check on the provided value.
7. In `sPOLController`, `_sellSPOLMulti` does not ensure that the total sold amount matches the requested amount, as done in `_buySPOLMulti`.

Code partially corrected:

- Items 1, 2 and 3 have been fixed with appropriate checks.
- Items 4, 5, 7 were accepted.
- Item 6 was accepted with a NatSpec warning added about 0 and excessive values.

7.6 No Slippage Protections

Informational **Version 1** **Acknowledged**

CS-POLYGON-SPOL-013

Across the system, none of the functions used to buy or sell sPOL implement slippage protections. Given the multiple ways the exchange rate could change (listed in L1 Exchange-Rate Appreciation Can Brick L2 Migrations or Enable Arbitrage), users could be exposed to significant slippage when buying or selling sPOL, if front-run by one of these actions. In particular, nothing prevents the various buy functions in the `sPOLController` and `sPOLChild` from minting 0 sPOL for a non-zero POL amount.

Acknowledged:

The Polygon team acknowledges the absence of slippage protections and states:

```
The exchange rate can only increase, so slippage protection is
not needed in the traditional sense. The only "sandwiching"
possible is restaking rewards right before a user buys, leading
to less sPOL — but of the same value in POL. For selling, a
rate increase is never worse for the seller.
```

7.7 Permit Signatures in Controller

Informational **Version 1** **Acknowledged**

CS-POLYGON-SPOL-031

The controller allows various operations to be performed via permit signatures for POL, sPOL, and validator shares.

It should be noted that:

- Anyone can front-run a controller operation that uses a permit signature by submitting the same permit directly to the token contract. The original transaction then fails when it tries to reuse the signature.
- Anyone can use a permit signature intended for a specific validator with a different validator, because the validator ID is not part of the signed data. Similarly, a signature intended for multi-validator operations can be used for single-validator operations (and vice versa). This should however not lead to a different outcome than the one intended by the signer.

Acknowledged:

No changes were made to permit-signature handling. Polygon states:

```
Acknowledged. The permit nonce check ensures consumption, and consumePermit
resets allowance to 0 after use. Front-running with the same permit just causes
a safe revert or successful user action. Validator selection is not important.
```

7.8 Withdrawal Amounts and Nonces Unsafe Casts

Informational **Version 1** **Acknowledged**

CS-POLYGON-SPOL-038

During sell operations, `sPOLController` explicitly casts withdrawal amounts to `uint128` and nonce values to `uint96` before persisting them to storage.

```
_addUserWithdrawNonceDetails(_user, _validator, uint128(dPOLAmount), uint96(userNonce));
```

This introduces a risk of silent truncation if the values exceed the maximum limits of these types (i.e., `amount > 2**128` or `validatorNonce > 2**96`). While unlikely under normal conditions due to the large limits, if truncation does occur it could lead to incorrect accounting, loss of funds, or other unintended behavior.

Acknowledged:

Polygon answered:

```
POL max supply (~11B with 18 decimals) requires 94 bits — fits
in uint128 with massive headroom. Validator nonces are sequential
counters that won't approach uint96 max (~7.9 x 10^28).
```

8 Notes

We leverage this section to highlight further findings that are not necessarily issues. The mentioned topics serve to clarify or support the report, but do not require an immediate modification inside the project. Instead, they should raise awareness in order to improve the overall understanding.

8.1 Admin Must Trigger Backfill for Users to Withdraw

Note Version 1

When users sell sPOL on L2 via `sellSPOL`, their withdrawal is assigned to the next backfill cycle (`backFillCycle + 1`):

```
UserOutstanding memory userOutstanding =
    UserOutstanding({outstandingPOL: polToReturn, backFillCycle: backFillCycle + 1});
```

Users can only withdraw their POL once their assigned backfill cycle is marked as completed in `withdrawPOL`. Since withdrawals are always assigned to the *next* cycle, users who sell sPOL cannot withdraw until the admin triggers another `_balanceWithL1()` call that starts and completes a new backfill cycle.

The admin must always trigger a subsequent backfill cycle to enable users to withdraw. Failure to do so will leave user funds locked indefinitely.

8.2 Inflated sPOL Supply

Note Version 1

sPOL sold on L2 is not burned immediately: it is first transferred to `sPOLChild` and tracked in `locallyToBeBurnedSPOL`. This creates a temporary mismatch where `totalSupply()` exceeds economically active sPOL.

That mismatch becomes persistent when reconciliation is completed via a local-settlement, because that path settles POL obligations without executing the L2 burn. Burning in that path is intentionally avoided to prevent creating a bridge exit flow to L1.

As a result, `totalSupply()` on L2 can remain inflated versus effective economic value, with excess sPOL retained in `sPOLChild` by design.

The Polygon team commented:

```
This is a limitation created by the bridge, so once we either update
the current bridge or switch to another one we will implement a proper
burn on migration and also clear the accumulated dead sPOL.
```

8.3 Integrators Gas Limits

Note Version 1

Across the system, several functions have O(N) gas costs where N is the number of withdrawals to process for the message sender. This is unlikely to be an issue for regular users, but integrator contracts that allow permissionless sPOL sells can hit the block/transaction gas limit if a user accumulates too many pending withdrawals. Integrators should cap outstanding withdrawals to avoid user-driven self-DoS from oversized batches.

8.4 Introducing New Cross-Chain Message Types

Note Version 1

L1-to-L2 messages use Polygon's state sync mechanism. If the child handler reverts, the message is stored in the `failedStateSync` mapping of `StateReceiver` and can be replayed permissionlessly until it succeeds. L2-to-L1 messages use merkle-proof exits. If the L1 handler reverts, the proof can be resubmitted.

In Version 2, unrecognized message types are silently discarded (the handler returns without reverting). Such messages cannot be replayed through the normal retry mechanisms.

When upgrading the tunnel contracts, any new message type that is dispatched before the receiving side supports it will be silently discarded. If the discarded message cannot be re-triggered (e.g., because it depends on the system state), an additional contract upgrade would be required to recover.

8.5 Native Token Transfer on Withdrawal

Note Version 1

`sPOLChild.withdrawPOL()` allows users to withdraw native POL. The function transfers value via `(bool success,) = payable(msg.sender).call{value: totalToWithdraw}("");`. Users should ensure their receiver contracts can accept native token transfers and do not have reentrancy guards or fallback logic that would cause the transfer to revert.

8.6 Permissionless Withdraw

Note Version 1

In the Controller, `withdrawPOL` can be called by anyone to trigger withdrawals for any user, without requiring prior approval from the user. This should be handled by integrator contracts to ensure that this does not break their expected flow.

8.7 `convertSPOLtoPOL(X)` When Total sPOL Supply Is Zero

Note Version 1

When the total sPOL supply is zero, `convertSPOLtoPOL(X)` returns `X`.

- Internally, the function is only called when selling sPOL, so the caller should revert if this case is reached and `X` is non-zero, since sPOL cannot be sold when none exists.
- Integrators should be aware of this edge case to avoid unexpected behavior.